InFEDge: A Blockchain-Based Incentive Mechanism in Hierarchical Federated Learning for End-Edge-Cloud Communications

Xiaofei Wang, *Senior Member, IEEE*, Yunfeng Zhao, Chao Qiu, *Member, IEEE*, Zhicheng Liu, Jiangtian Nie, *Member, IEEE*, and Victor C. M. Leung, *Life Fellow, IEEE*

***Abstract*— Advances in communications and networking technologies are driving the computing paradigm toward the end-edge-cloud collaborative architecture to leverage ubiquitous data and resources. Opposite to centralized intelligence, Hierarchical Federated Learning (HFL) relieves overwhelmed communication overhead and enjoys the advantages of high bandwidth as well as abundant computing resources while retaining privacy-preserving benefits of Federated Learning (FL). It is difficult to balance system overhead and model performance in the HFL framework, while it could be solved by introducing an incentive mechanism. Although the incentive mechanism can alleviate the above anxiety by compensating relevant participants, some limitations (*multi-dimensional properties, incomplete information and unreliable participants*) will significantly degrade the performance and efficiency of the designed mechanism. To address the challenges caused by the above limitations, we propose InFEDge, a blockchain-based incentive mechanism in the HFL. The InFEDge considers 1) *multi-dimensional individual properties* to model system participants and proves the uniqueness of Nash equilibrium with the closed-form solution. Meanwhile, 2) we transform the problem under *incomplete information* into a contract game where we obtain the optimal solution. Moreover, 3) we also leverage the blockchain to provide economic incentives, prevent *unreliable participants*' disturbance and further ensure data privacy by implementing the mechanism in the smart contract to offer a credible, faster, and transparent resource trading system. Experimental evaluations on a proof-of-concept testbed along with real traces demonstrate the superiority of our mechanism. Further, our method solves a real-world user allocation problem for future communications and networking.**

***Index Terms*— Federated learning, incentive mechanism, blockchain, edge intelligence, game theory.**

Manuscript received 16 March 2022; revised 16 June 2022; accepted 30 June 2022. Date of publication 13 October 2022; date of current version 22 November 2022. This work was supported in part by the National Key Research and Development Program of China under Grant 2019YFB2101901, in part by the National Natural Science Foundation of China under Grant 62072332, in part by the National Natural Science Foundation of China (Youth) under Grant 62002260, in part by the China Postdoctoral Science Foundation under Grant 2020M670654, in part by the Open Research Fund from Guangdong Laboratory of Artificial Intelligence and Digital Economy (Shenzhen, SZ) under Grant GML-KF-22-03, and in part by Research and Innovation Project for Postgraduates in Tianjin (Artificial Intelligence) under Grant 2021YJSO2B06. *(Corresponding author: Chao Qiu.)*

Xiaofei Wang and Chao Qiu are with the School of Computer Science and Technology, College of Intelligence and Computing, Tianjin University, Tianjin 300350, China, and also with the Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen 518000, China (e-mail: xiaofeiwang@tju.edu.cn; chao.qiu@tju.edu.cn).

Yunfeng Zhao and Zhicheng Liu are with the School of Computer Science and Technology, College of Intelligence and Computing, Tianjin University, Tianjin 300350, China (e-mail: yfzhao97@tju.edu.cn; liuzhicheng@tju.edu.cn).

Jiangtian Nie is with the School of Computer Science and Engineering, Nanyang Technological University, Singapore 639798 (e-mail: jnie001@e.ntu.edu.sg).

Victor C. M. Leung is with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China, and also with the Department of Electrical and Computer Engineering, The University of British Columbia, Vancouver, BC V6T 1Z4, Canada (e-mail: vleung@ieee.org).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSAC.2022.3213323.

Digital Object Identifier 10.1109/JSAC.2022.3213323

I. Introduction

RECENTLY, with the rapid development of communications and networking technologies [1], more and more devices and intelligent applications have emerged on the edge of networks. Due to the high penetration of devices, there is an exponential increase of data generated at network edge devices [2], [3]. Big data and improving computing capacity have prompted the development of Artificial Intelligence (AI) and deep learning and revolutionized our life [4]. It also implies the coming of the intelligent era of ubiquitous data generation, processing and device deployment in end-to-end cloud systems. For example, massive data is usually aggregated and stored in the cloud, together with AI algorithms, to enable multifarious intelligent services. However, significant privacy leakage and communication load issues arise when delivering sensitive raw data to the cloud via network transmission.

As an emerging computing paradigm, the end-edge-cloud collaborative computing architecture fits perfectly with end-edge-cloud systems, revolutionizing the current cloud computing and cloud-edge architecture [5]. The ubiquitous computing resources from clouds, edges, and ends work together via the networking function, thus deriving the deep integration of computing and networking [6]. The end-edge-cloud collaborative computing architecture has become the mainstream computing architecture and is used in various fields, e.g., computer vision, smart city and healthcare [7].

As a compelling training framework of the end-edge-cloud collaborative computing architecture, Hierarchical Federated Learning (HFL) framework is advocated by some studies [8], [9], [10], [11], which involves multi-layers of model aggregations. As depicted in Fig. 1, HFL combines the advantages of centralized learning and FL, which not only reduces the com-

0733-8716 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

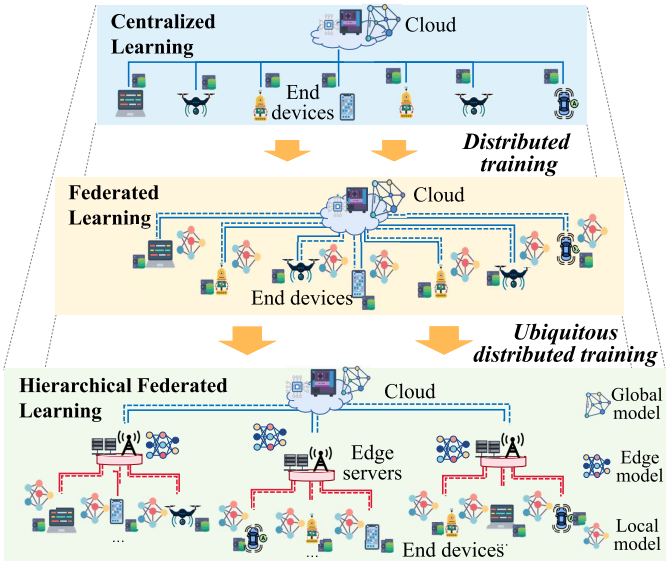


Fig. 1. Centralized learning v.s. Federated learning v.s. Hierarchical federated learning. The HFL framework fits perfectly with end-edge-cloud systems, revolutionizing the current cloud computing and cloud-edge architecture.

munication overhead in a wireless local area network (WAN), but also improves training efficiency and accuracy [12]. As a promising evolution of Federated Learning (FL) [13], HFL inherits the property of protecting data privacy while enabling distributed training on massive end devices (a.k.a., client). Further, it leverages its advantages to train the global model with high accuracy and fast convergence at the lower layer, benefiting from high bandwidth and sufficient computation resources at the edge servers. Intuitively, HFL can avoid frequent cloud aggregations without performance degradation and be quickly applied in various fields (e.g., industrial or Internet scenarios) for AI-based services. Moreover, coordination by the edge servers in proximity allows for more efficient communication and computation resource allocation among clients.

Although HFL promises several benefits, numerous nontrivial issues in the current HFL method prevent it from being a generic platform for different services and applications. For example, massive model parameters need to be exchanged in multiple layers and multiple update iterations in a typical HFL system, regardless of synchronous update [14] and its variants [15] or asynchronous update [16]. Due to the high real-time and reliable communication requirements of the future scenarios, the networks with a high communication load will be overwhelmed by the large number of parameter uploads from clients in HFL. Further, training models in HFL systems at corresponding nodes consume multiple limited resources and face heavy communication and computation overheads. The extreme operations of greatly reducing the number of parameter uploads or data training will significantly decline model performance. Hence, how to strike an optimal balance between system overhead and model performance remains to be tackled in the HFL system.

By compensating the associated nodes differently for using their different resources, the incentive mechanism [17] can balance overhead (e.g., communication and computation overheads) and model performance. And it also removes the impractical assumption that the related nodes voluntarily participate in computing without asking for any returns. We plan to propose an effective and credible incentive mechanism in the HFL framework. We need to model and incentivize the cloud, edge servers, and clients to achieve satisfactory training results. However, the limitations in incentive mechanism design have not been well-addressed in the newest HFL framework and still face some challenges.

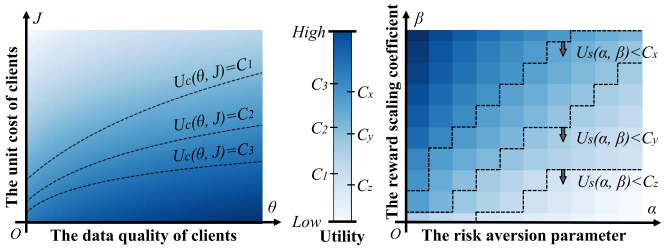


Fig. 2. Cloud's preference order is affected by various factors, including (a) clients' data quality and training cost, (b) edge servers' two-dimensional individual properties, i.e., the risk aversion and the reward scaling.

A. Multi-Dimensional Individual Properties

The training efficiency has long been challenged by the heterogeneity of communication conditions, computing capabilities, and datasets. Considering any one of them is insufficient to guarantee long-term system efficiency and stability. As shown in Fig. 2, the cloud's utility varies with clients' properties and edge servers' participating willingness [18]. Unfortunately, many current mechanisms focus on one dimension of heterogeneity without considering the tradeoff between data and resource heterogeneity. Hence, *How to jointly incentivize heterogeneous clients and edge servers with multi-dimensional individual properties for HFL?*

B. Incomplete Information

Under incomplete information of individual properties, designing an incentive mechanism that encourages system participants to participate in the training process is difficult to achieve [19], [20]. The main reason is that individual properties are private and often difficult to disclose publicly, e.g., each client's training cost depends on its computing power, and the data quality depends on its collecting characteristic. Then, *how to achieve incentive effects and to stimulate clients and edge servers to participate efficiently and truthfully under incomplete information?*

C. Unreliable Participants

Some participants may unintentionally perform undesirable behaviors and deviate from the learning protocol to mislead a global model training of a federated learning task or incur important information leakage. Further, malicious participants deviate from the learning protocol and send corrupt local updates to poison the global model [21]. Since there are more roles in HFL than the traditional one, it will be more difficult

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

to detect if multiple roles collude and affect overall system performance. *How to offer these heterogeneous participants deployed by distinct companies and individuals a decentralized and transparent resource trading system?* As a new paradigm of information sharing, blockchain is an optimal candidate for enabling incentive mechanisms in HFL. The related information and transactions can be recorded in the block without tamperproof, and thus the revenues of the participants can be guaranteed [22], [23], [24], [25], [26], [27], [28].

To cope with the above challenges, **we design and implement a blockchain-based incentive mechanism, InFEDge, to validate the capability to maximize the participants' valuation and guarantee public authority and fairness.** The major contributions of this article are shown as follows.

- We propose a game-based incentive mechanism that models the rationality of clients, edge servers and the cloud in the presence of multi-dimensional individual properties. The existence and uniqueness of Nash equilibrium under complete information are proved with the closed-form solution.
- We transform the incentive issue into a hierarchical contract problem and obtain the optimal solution under the incomplete information of multi-dimensional individual properties. Blockchain is introduced to implement the incentive mechanism in the smart contract and provide a credible, faster and transparent environment for the system.
- We evaluate InFEDge on a proof-of-concept testbed with real traces to demonstrate the superiority of our mechanism, e.g., InFEDge has 21.2% and 32.6% higher utilities than baselines for cloud and clients, respectively. And we also use it to solve a real-world user allocation problem for future communications and networking. Moreover, we find the pursuit of the cloud's utility is achieved by enhancing clients' utilities with appropriately impacting edges' utilities.

The rest of this paper is organized as follows. Section II reviews the related work. Section III discusses the system model. Section IV and Section V elaborate the InFEDge under complete and incomplete information. Section VI presents the experiment results of the proposed scheme. Section VII shows the use case of InFEDge for 5G resource optimization. Section VIII concludes the paper.

II. Related Work

A. FL or HFL in Cloud and Edge Computing

How to optimize the system performance of FL has attracted a lot of attention in areas of communication efficiency [29], convergence preserving [30], and training speedup [31]. To reduce the bits on gradient exchanges in FL, Ahn et al. [32] utilize techniques, such as neural network pruning [33], weight quantization [34], message sparsification [29] and knowledge distillation [32] to compress the ML model to achieve transmitting less information while maintaining the high learning performance. Different from using the ubiquitous cloud to realize model aggregation in FL, some researchers take advantage of edge computing to empower federated learning [13], [35], [36], [37]. On the other hand, some works only consider the logical topology in distributed learning but not with any co-computing architecture [38], [39], which may hinder the application in the current mobile network.

To further improve communication efficiency, the HFL framework is proposed by introducing an edge layer, which leverages edge nodes as intermediaries to perform partial model aggregation with efficient client-edge communication. Thus, it relieves core network transmission overhead in the cloud server [40]. For example, Liu et al. propose a HierFAVG algorithm that performs two levels of synchronous model aggregation by extending the conventional FedAvg algorithm to the hierarchical setting [14]. Chai et al. present FedAT, a novel federated learning method that synergistically combines synchronous intra-tie training and asynchronous cross-tier training to improve the convergence speed and reduce communication cost [41]. These methods improve the convergence speed and reduce communication costs of end devices in the edge computing system. Nevertheless, the selfless contribution characteristics are not consistent with the actual situation of edge computing. Although HFL has some advantages, the above studies do not consider the application of individual rational HFL in edge computing.

B. Incentive Mechanism for Federated Learning

Though many works consider selecting high-quality clients to improve system performance [42], some of them assume that clients selflessly contribute their data, computing and communication resources to the training process. From a rational point of view, most clients are resource-constrained and reluctant to participate in the training process. For this reason, several works have studied the FL with incentive mechanism [43]. E.g., Chu et al. [44] take multi-dimensional individual properties into account and present an incentive mechanism FMore with a multi-dimensional procurement auction of K winners. Ding et al. [19] present an analytical study on the server's optimal incentive mechanism design in the presence of users' multi-dimensional private information (e.g., training cost and communication delay). However, they do not specifically consider the incentive mechanism for the HFL.

Kang et al. [45] introduced reputation as the metric to measure the reliability of the mobile devices and proposed a contract theory to incent mobile devices to participate in the FL task. The numerical result shows that this scheme is efficient in selecting reliable mobile devices and can improve the efficiency of FL. Zhan et al. [46] proposed a two-stage Stackelberg game model as the incentive mechanism for FL. The server announces a total reward while each edge node determines its training strategy to maximize its own utility. Then, the authors propose the deep reinforcement learning-based incentive mechanism. However, they only focus on one dimension of heterogeneity, which is insufficient to guarantee long-term system efficiency and stability and does not apply to more general multi-layer network architectures, e.g., end-edge-cloud collaborative networks. Further, we lack an incentive mechanism that comprehensively considers multiple factors,

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

e.g., multi-dimensional individual properties, incomplete information and unreliable participants.

C. Blockchain for Federated Learning

The main advantage of blockchain is that it has excellent information security capabilities, prompting many researchers to use it to alleviate privacy anxiety. Some studies have used blockchain to enhance system security mechanisms by detecting malicious behavior [47], [48]. Shayan et al. [47] proposed a decentralized public peer-to-peer system named Biscotti that co-designs a privacy-preserving FL process using blockchain. The task publisher can detect the poisoning attack by comparing the influence of with and without the model update on a database. Fung et al. [48] proposed the FoolsGold scheme to identify malicious edge nodes based on the principle that honest edge servers can be separated from malicious edge nodes by the diversity of their gradient updates. The malicious edge nodes can be detected with the cosine similarity method as their gradient updates are more similar to each other than honest edge nodes.

Another critical blockchain application is an attractive tool to facilitate transparent economic mechanism designs. Some use cases towards incentive mechanisms for blockchain-based FL [49], [50], [51] pay attention to providing economical solutions. They realize their desired objectives by designing a rigorous reward policy, fully integrating blockchain security with a reasonable and efficient incentive mechanism. Toyoda and Zhang [49] implement a rigorous reward policy design for blockchain-based by providing an economical solution to realize desired objectives under the rational assumption of mobile users. Ur Rehman et al. [50] design an integrated blockchain-based decentralized reputation system to ensure trustworthy collaborative model training in edge computing environments. Somy et al. [51] configure some system chaincode functions to incentivize all clients to record their actions in the distributed ledger to help blockchain verify users' behaviors in FL training. However, none of these efforts integrates blockchain and incentive mechanisms into the HFL under the end-edge-cloud collaborative architecture.

III. System Model

A. System Overview

The framework of HFL in end-edge-cloud orchestrated systems is composed of 1 cloud, L edge servers, and N clients.

1) Cloud: The cloud publishes a training task in HFL, the purpose of which is to obtain a satisfactory model performance and earn the corresponding bonus. Therefore, the cloud will deliver the gain to edge servers and indirectly encourage clients to participate in model training.

2) Edge Servers: There is a set of edge servers $\mathcal{L}$, in which each edge server $l \in \mathcal{L}$ is connected to a set of clients S_l. We assume that each client is connected to one edge server and remains unchanged during training. As an intermediate component of the HFL, edge servers need to aggregate model parameters and balance the gain between the cloud and the connected clients.

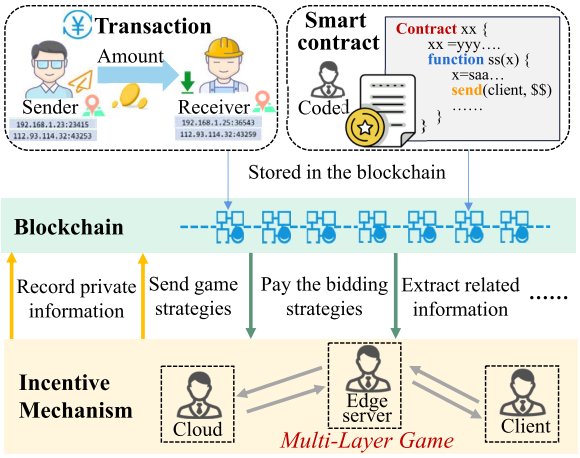


Fig. 3. Blockchain-based incentive mechanism framework.

3) Clients: There is a set of clients $\mathcal{N}$ who are expected to collaboratively learn a global model using their data. Each client $n \in \mathcal{N}$ connects to the edge server $\mu(n)$. Hence, we have $n \in S_{\mu(n)}$ and denote $\mathbf{S}_n = S_{\mu(n)} \subseteq \mathcal{N}$. Each client maintains a dataset $\mathcal{X}_n$ with data quality θ_n where θ_n can be obtained from the model accuracy measured with the local dataset (i.e., local accuracy). Similar to reference [52], we assume that $\theta_n = -\eta/\log(\epsilon_n)$, where ϵ_n is the local accuracy of client n and η represents the data quality coefficient.

4) Blockchain: Blockchain enables a shift in the computing paradigm from centralized control to decentralized control [22], [53], [54], [55] while it permanently records related information (e.g., multi-dimensional individual properties) and transactions between two parties without the need for third-party authentication. Blockchain provides a feasible solution for averting unreliable participants and preserving privacy which avoids the computational overhead and system complexity introduced by utilizing specific privacy-preserving cryptographic algorithms, such as homomorphic encryption.

This motivates us to introduce a blockchain-based incentive mechanism that guarantees public authority and fairness, as illustrated in Fig. 3. Transactions are stored in blocks containing timestamps and references and growth as a chain. Blockchain inspires us to build incentive, transparency, and fairness for our work, with its incentive feature and smart contracts supporting Turing-complete programmability. The auto-running property enabled by smart contracts offers an efficient interaction approach. Building the incentive mechanism into the blockchain platform becomes an effective way to tackle challenges of engaging self-interested, malicious participants, privacy leakage, etc.

5) Learning Process in HFL: We consider the HFL framework as Fig. 4, in which one training model goes through model aggregation in the edge server and the cloud layers. Therefore, the shared model parameters by clients in a global iteration involve edge aggregation and cloud aggregation. The global model ω is learned by minimizing the overall empirical risk of the loss $F(\omega)$ on the union of all local datasets. After every τ_w local update on each client, each edge server aggregates model parameters from connected clients. For every

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

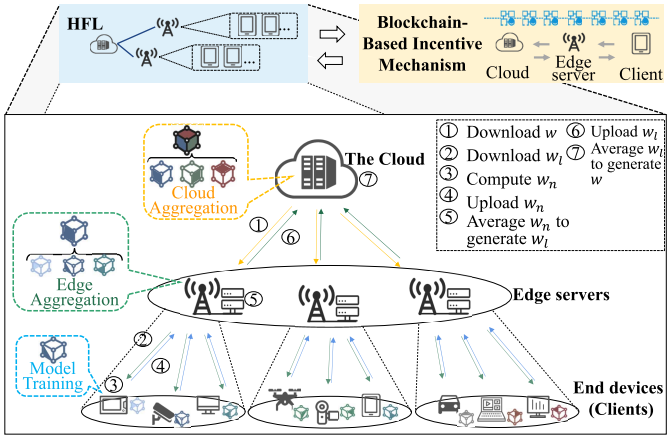


Fig. 4. Hierarchical federated learning process and data flows.

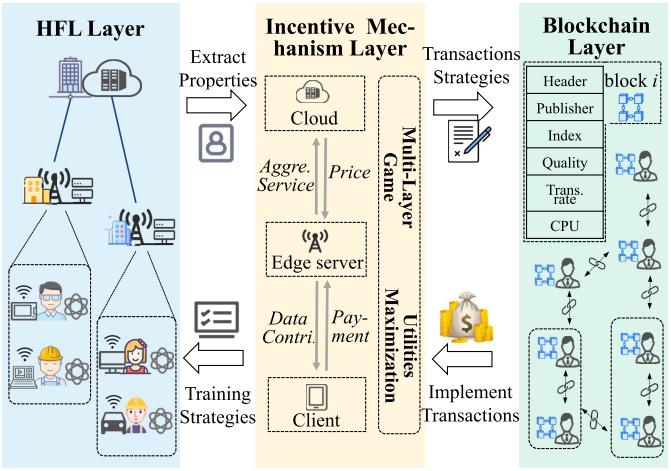


Fig. 5. The whole InFEDge system with three layers, which are HFL layer, the incentive mechanism layer and the blockchain layer.

τ_e edge model aggregations, the cloud receives and aggregates model parameters from edge servers. We assume that the cloud aggregates τ_c times in each round of training and denote $\omega_n(t)$ as the local model parameters after the t-th local update. The evolution of local model $\omega_n(t)$ is given as in (1), shown at the bottom of the next page.

6) The Blockchain-Based Incentive Mechanism for HFL: As shown in Fig. 5, we introduce blockchain in HFL and incentive mechanism system to ensure the overall system's fairness, transparency, and credibility. The whole system, named InFEDge, has three layers: the HFL layer, the incentive mechanism layer, and the blockchain layer. Blockchain in the whole InFEDge system is used to verify key parameters and manage the economic incentives of participants guided by the incentive mechanism system, and only the related data can remain on the distributed ledger. The functions of each layer in the whole InFEDge system are shown as follows.

(1) **HFL Layer:** Participants in the HFL layer upload their key parameters updates (e.g., data quality, communication conditions, and computing capability) to the blockchain. In addition, it is more important to execute the model training according to the guidance strategies from the incentive mechanism and the economic incentives.

(2) **Blockchain Layer:** Blockchain not only verifies and maintains the related updates but also implements the issuance of economic incentives based on the output strategies of the incentive mechanism. We use smart contracts in blockchain to implement the logics in the proposed mechanism. The interactions take the form of transactions and are stored in the blockchain, which offers a privacy-preserving and efficient solution.

(3) **Incentive Mechanism Layer:** In the incentive mechanism layer, the relevant strategies are calculated by reading the corresponding parameters so as to guide the training process of HFL and how to provide proper economic incentives.

The whole InFEDge system enables better management of participants, ensures the quality and quantity of data contributed during model training and improves the performance of the model. Additionally, it also prevents malicious participants from attacking the system and privacy leakage.

B. Client Modeling

1) Client Gain: After receiving the HFL task published by the cloud, the client will contribute some data with a size of x_n to train the local model. If $x_n = 0$, client n does not participate in the model training during this round. Client n's gain depends on the other clients' data contribution in the set $\mathbf{S}_n$, i.e., $x_{-n} = (x_i, \forall i \in \mathbf{S}_n \backslash \{n\})$ and the gain $R_{\mu(n)}$ paid by the edge server $\mu(n)$. Note that clients in $\mathbf{S}_n$ will allocate $R_{\mu(n)}$ according to the proportion p_n and $\sum_{n \in \mathbf{S}_n} p_n = 1$. Similar to [8] and [46] and without loss of generality, we have

$$p_n = \theta_n x_n / (\theta_n x_n + \sum_{i \in \mathbf{S}_n \backslash \{n\}} \theta_i x_i). \tag{2}$$

Therefore, the gain of client n can be described as

$$f(x_n, x_{-n}) = p_n R_{\mu(n)}, \tag{3}$$

where $f(\cdot)$ is non-decreasing and concave with decreasing marginal satisfaction, indicating the decreasing marginal preference of clients to content.

2) Client Cost: The training cost of the client includes computational cost and collection cost, which are proportional to the amount of data used for training. Let C_E denote the cost of unit data collected by the client. The client n's CPU cycle frequency, the number of CPU cycles for training unit data and the effective capacitance parameter of the computing chipset are denoted by f_n, c_n and ζ. Hence, the energy consumption and computation time [56] for one local iteration are

$$\begin{aligned} E_n(x_n) &= \zeta c_n x_n f_n^2, \\ T_n(x_n) &= c_n x_n f_n^{-1}. \end{aligned} \tag{4}$$

We consider that the individual properties of participants are fixed during a single game. Therefore, the training cost of client n in one round is represented as

$$cost_n^E = (\lambda_e E_n(x_n) + \lambda_t T_n(x_n)) \tau_c \tau_e \tau_w + C_E x_n, \tag{5}$$

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

where λ_e and λ_t are weighting parameters of energy and delay for client training requirements. For ease of discussion, we set client n's training cost as $cost_n^E = J_n x_n$ where J_n is the unit training cost.

3) Client Types: Many current mechanisms focus on heterogeneity in only one dimension without considering the tradeoffs between data and resource heterogeneity. Hence, motivated by Fig. 2, we consider that clients are distinguished by *two-dimensional individual properties*: the training cost and the data quality. For the convenience of presentation, we refer to a client with $\pi_k^E = (\theta_k, J_k)$ as a type-k client. All clients belong to a set $\mathcal{K} = \{1, 2, \ldots, K\}$ with K types. Each client is a type-k client with the probability ρ_k and $\sum_{k=1}^{K} \rho_k = 1$. We can transform clients' two-dimensional individual properties into a *one-dimensional criterion*, $\sigma_k = \theta_k / J_k$, which indicates the system's preference for different client types. Thereby, the general form of the client n's *utility*, i.e., the difference between the client's gain and cost, can be formulated as

$$u_n^E(x_n, x_{-n}) = f(x_n, x_{-n}) - J_n x_n. \tag{6}$$

C. Edge Server Modeling

1) Edge Server Gain: Similar to clients, the gain of each edge server hinges on the gain R_C paid by the cloud and the performance of the other edge servers. The edge server l will allocate R_C on a proportion p_l, which is similar to clients and can be defined as

$$p_l = \sum_{n \in S_l} \theta_n x_n / (\sum_{n \in \mathcal{N}} \theta_n x_n). \tag{7}$$

Hence, without loss of generality, the gain of edge server l is

$$h(R_l) = \ln(\alpha_l + p_l R_C), \tag{8}$$

where the function $h(\cdot)$ is defined as a risk-averse model [57], [58] and can also be extended to more sophisticated expressions. α_l is a risk aversion parameter [18], representing the edge server's risk of participating in the HFL task from the perspective of the gain. The larger the α_l is, the larger the risk is and the lower the willingness.

We innovatively introduce a risk aversion model to model participants more realistically. Risk measures used in optimization problems allow mitigating the effects of inexact knowledge. They bridge the gap between conservative worst-case approaches and stochastic approaches. The modelling of intermediate aggregators is generally based on the assumption that participation in training is risk-free, but this assumption does not always hold. In reality, the role of an aggregator is similar to that of a middleman coordinating between buyers and sellers, so there should be more focus on risk considerations. For example, when edge servers come from third-party services and do not belong to the same alliance with the cloud, edge servers will pay more attention to risk aversion and have a low willingness to participate in HFL. Therefore, the risk-averse model is suitable for practical implementations with inexact knowledge.

2) Edge Server Cost: The cost of the edge server includes communication costs and coordination costs. Denote C_L, r_l, P_l^T and D as the unit coordination cost, the edge server l's achievable transmission rate, the transmission power and the data size of model parameters, respectively. According to [56], $T_l = D/r_l$ and $E_l = P_l^T T_l = D P_l^T / r_l$ are the delay and energy consumption for communication. Therefore, the cost of the edge server l in a round is

$$cost_l^S(R_l) = \frac{R_l}{\beta_l} + \tau_c \tau_e (\lambda_e E_l + \lambda_t T_l) + C_L |S_l|, \tag{9}$$

where $|\cdot|$ represents the cardinality of a set. β_l is a reward scaling coefficient, indicating the willingness to participate in the HFL task from the perspective of the cost. The larger the β_l is, the higher the willingness is.

3) Edge Server Types: Edge servers are distinguished by *two-dimensional individual properties*: the risk aversion parameter α and the reward scaling coefficient β. For the convenience of presentation, we refer to an edge server with $\pi_q^S = (\alpha_q, \beta_q)$ as a type-q edge server. All of them belong to a set $\mathcal{Q} = \{1, 2, \ldots, Q\}$ with Q types. Each edge server is a type-q edge server with the probability ρ_q and $\sum_{q=1}^{Q} \rho_q = 1$. Similar to clients, we can transform edge servers' two-dimensional individual properties into a *one-dimensional criterion*.

The general form of the edge server l's *utility*, i.e., the difference between the edge server's gain and cost, can be formulated as

$$u_l^S(R_l) = h(R_l) - cost_l^S(R_l). \tag{10}$$

D. Cloud Modeling

The gain of the cloud is determined by the performance of the global model (related to the global accuracy). We conduct experiments to measure the model accuracy under different amounts of training data and show the results in Fig. 6. The test accuracy can be regarded as a concave function with respect to the amount of training data [46]. Therefore, without loss of generality, the gain of the cloud is defined as a concave function $\lambda \cdot g(\sum_{n \in \mathcal{N}} \theta_n x_n)$ [59], [60], a.k.a., the model performance. $\lambda > 0$ is a system parameter.

The cloud's cost is the total gain R_C allocated to clients and edge servers. The total gain can be rewritten as

$$\omega_n(t) = \begin{cases} \omega_n(t-1) - \eta_t \nabla F_n(\omega_n(t-1)), & t | \tau_w \neq 0, \\ \frac{\sum_{n \in \mathbf{S}_n} x_n [\omega_n(t-1) - \eta_t \nabla F_n(\omega_n(t-1))]}{\sum_{n \in \mathbf{S}_n} x_n}, & t | \tau_w = 0, \\ t | \tau_w \tau_e \neq 0, & \\ \frac{\sum_{n \in \mathcal{N}} x_n [\omega_n(t-1) - \eta_t \nabla F_n(\omega_n(t-1))]}{\sum_{n \in \mathcal{N}} x_n}, & t | \tau_w \tau_e = 0. \end{cases} \tag{1}$$

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

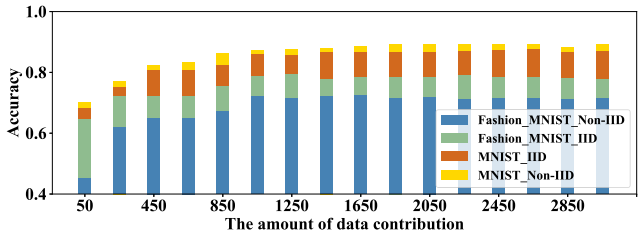


Fig. 6. Using HFL to test accuracy with MNIST and Fashion-MNIST datasets under the independent identically distributed (IID) and Non-IID settings.

$R_C = \sum_{n\in\mathcal{N}} \theta_n x_n P$ where P is the unit service price and is determined by the cloud. To summarize, the general form of the cloud's *utility* in a round is formulated as

$$u^C(P) = \lambda \cdot g(\sum_{n\in\mathcal{N}} \theta_n x_n) - R_C. \tag{11}$$

E. Information Conditions

To study the impact of information conditions, we study incentive mechanisms under two information conditions. **1) Under Complete Information:** The participants' individual properties (or types) are public information in the system. However, this case is not practical due to security risks brought by exposing individual properties. **2) Under Incomplete Information:** The participants have incomplete information about the individual properties of each other while knowing the distributions of individual properties (or types).

IV. Incentive Mechanism Under Complete Information

In this section, we model the interactions among the cloud, edge servers and clients as *a multi-layer Stackelberg game* under complete information, as illustrated in Fig. 7. The multi-layer end-edge-cloud architecture is mapped to subgames interconnected through utility functions that describe realistic price responses at each layer. The participants in the higher layer act as the leaders to decide all the gain allocation for participants in the lower layer to maximize their utilities. The participants in the lower layer then act as the followers to decide their strategies. In this scenario, all nodes, i.e., the cloud, edge servers and clients, have already been registered on a permissioned blockchain.

We aim to find the optimal strategy profile in the game. These will be recorded and also performed on the blockchain through a smart contract. In the following, the subgame perfect Nash equilibrium is used for solving the multi-layer Stackelberg game problem. The main notations are summarized in Table I.

Definition 1 (Nash Equilibrium): The strategy profile (x_n^*, x_{-n}^*) *is a Nash equilibrium (NE) in the game if satisfying*

$$u_n^E(x_n^*, x_{-n}^*) \geq u_n^E(x_n, x_{-n}^*), \tag{12}$$

for any $x_n > 0$.

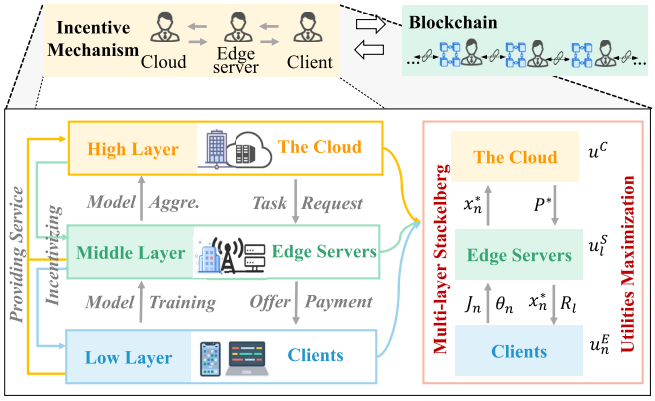


Fig. 7. The multi-layer Stackelberg game in the HFL framework.

A. Optimal Strategy in Client's Subgame

First, we demonstrate the existence and uniqueness of the NE in a general sense, and second, we show how to select appropriate clients that can generate the NE strategy profile to participate in the HFL task.

For a given strategy profile x_{-n}, if a strategy x_n^* maximizes $u_n^E(x_n, x_{-n})$ over all $x_n > 0$, it is the optimal strategy for client n. Hence, the optimal strategy for each client is the solution to the following utility optimization problem,

$$\begin{aligned} \textbf{P1:}\ & \max_{x_n} u_n^E(x_n, x_{-n}) \\ & s.t.\ \ 0 \leq x_n \leq |\mathcal{X}_n|. \end{aligned} \tag{13}$$

According to Definition 1, we conclude that a NE is one in which every client plays its optimal strategy.

Theorem 1 (Existence and Uniqueness of NE for Clients): A NE exists in the clients' data contribution subgame.

Proof: First, the strategy space of all clients can be defined as a non-negative, non-empty and compact subset of Euclidean space. Further, we compute the first-order and second-order derivatives of utility $u_n^E(x_n, x_{-n})$ with respect to x_n:

$$\frac{\partial u_n^E(x_n, x_{-n})}{\partial x_n} = \frac{\theta_n \sum_{i\in\mathbf{S}_n\setminus\{n\}} \theta_i x_i}{(\sum_{i\in\mathbf{S}_n} \theta_i x_i)^2} \cdot R_{\mu(n)} - J_n, \tag{14}$$

and $\partial^2 u_n^E(x_n, x_{-n})/\partial(x_n)^2 < 0$. Since the second-order derivative of $u_n^E(x_n, x_{-n})$ is not less than zero, the client's utility is a concave function in all x_n. Therefore, there is a NE in the clients' data contribution subgame [61]. Meanwhile, since the second-order derivative is also less than zero, the utility $u_n^E(x_n, x_{-n})$ is a strictly concave function and its maximum value is unique. Thus, we conclude that the value that maximizes the client's utility function is the optimal strategy for the client and is a unique and stable NE. □

If the edge server l's strategy R_l and other clients' strategies x_{-n} are given, the optimal strategy can be obtained by setting the first-order derivative as zero and denoted by, as in (15), shown at the bottom of the next page.

Next, we discuss how to select appropriate clients to participate in the training task in the following theorem:

Theorem 2 (Clients Selection): For at least two clients in the clients' data contribution subgame, the clients can be sorted

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SYSTEM NOTATIONS

Notation	Description
x_n, x_{-n}	Strategies of n and clients which in $\mathbf{S}_n \backslash \{n\}$
R_l, R, P	l's, all edge servers' and the cloud's strategies
x_n^κ, R_l^κ, P^κ	n's, l's and cloud's κ-th strategies in the game process
x_n^*, R_l^*, P^*	n's, l's and the cloud's optimal strategies
$\mathcal{X}_n$, θ_n, D	n's dataset and data quality and the size of training model
τ_c, τ_e, τ_w	Number of model updates of the cloud, edge servers and clients
f_n, c_n, ζ,	n's CPU cycle frequency, CPU cycles for unit data and chipset's effective capacitance parameter
p_n, p_l	Proportions of n and l for allocating the gain
$\mathcal{N}$, $\mathcal{L}$	The sets of all the clients and edge servers
S_l, ϵ_n	Clients connected to edge server l and n's local accuracy
K, Q	Number of client's types and edge server's types
α_l, β_l	l's risk aversion parameter and reward scaling coefficient
ρ_k, ρ_q	Probability of type-k client and type-q edge server
P_l^T, r_l	l's transmission power and achievable transmission rate
$\mu(n)$, J_n	n connected edge server and n's training cost
λ_e, λ_t	The weighting parameters of energy and delay
σ_k, π_q^S	one-dimensional criterion of type-k client and type-q edge server
C_E, C_L	Edge server's unit coordination cost and client's unit collection cost
$\mathcal{E}$, G	Allocation strategy and bandwidth of 5G networks
T, T_0	The system delay and maximum threshold in the use case
D_u, o_u^X, o_u^Y	User u's data loads and two-dimensional coordinates

in ascending order according to their data quality and training cost. There exists a set of clients $\mathbf{S}_n' \subseteq \mathbf{S}_n$ where

$$J_n/\theta_n < \sum_{i\in\mathbf{S}_n'} (J_i/\theta_i)/(|\mathbf{S}_n'|-1) \tag{16}$$

ensures

$$x_n^* = \begin{cases} \frac{(|\mathbf{S}_n'|-1)R_{\mu(n)}}{\theta_n \sum_{i\in\mathbf{S}_n'} J_i/\theta_i}\left(1 - \frac{(|\mathbf{S}_n'|-1)J_n/\theta_n}{\sum_{i\in\mathbf{S}_n'} J_i/\theta_i}\right), & n \in \mathbf{S}_n', \\ 0, & n \notin \mathbf{S}_n'. \end{cases} \tag{17}$$

Proof: (i) If $|\mathbf{S}_n'| = 0$, a non-participating client can increase its utility from zero to $R_{\mu(n)}/2$ if the client unilaterally changes x_n from zero to $R_{\mu(n)}/2J_n$. This contradicts with the assumption of NE. Thus, the condition $|\mathbf{S}_n'| = 0$ does not hold and we have $|\mathbf{S}_n'| \geq 1$.

(ii) If $|\mathbf{S}_n'| = 1$, a participating client n has $x_n > 0$ and other clients' strategies $x_i = 0$ for all $i \in \mathbf{S}_n \backslash \{n\}$. Therefore, the utility of client n equals $(R_{\mu(n)} - J_n x_n)$. In this case, client n can increase its utility by changing its data contribution from x_n to $x_n/2$, which contradicts with the uniqueness of the NE. Thus, the condition $|\mathbf{S}_n'| = 1$ does not hold and we have $|\mathbf{S}_n'| \geq 2$.

(iii) $|\mathbf{S}_n'| \geq 2$. From setting the first-order derivative of the utility $u_n^E(x_n, x_{-n})$ as zero, we can derive

$$(|\mathbf{S}_n'|-1)R_{\mu(n)} = \sum_{i\in\mathbf{S}_n'} \theta_i x_i \sum_{i\in\mathbf{S}_n'} J_i/\theta_i, \tag{18}$$

which can be rewritten as:

$$\sum_{i\in\mathbf{S}_n} \theta_i x_i^* = \frac{(|\mathbf{S}_n'|-1)R_{\mu(n)}}{\sum_{i\in\mathbf{S}_n} J_i/\theta_i}. \tag{19}$$

Combining Eq. (15) and Eq. (19) and summing $\mathbf{S}_n' \backslash \{n\}$ give the following relationships among the client n and the other clients in $\mathbf{S}_n'$:

$$\sum_{i\in\mathbf{S}_n'\backslash\{n\}} \theta_i x_i^* = \frac{(|\mathbf{S}_n'|-1)^2 R_{\mu(n)} J_n/\theta_n}{(\sum_{i\in\mathbf{S}_n'} J_i/\theta_i)^2}, \tag{20}$$

and $J_n/\theta_n \sum_{i\in\mathbf{S}_n'} \theta_i x_i^* \geq R_{\mu(n)}$.

Since $J_n \sum_{i\in\mathbf{S}_n\backslash\{n\}} x_i^* = J_n \sum_{i\in\mathbf{S}_n'\backslash\{n\}} x_i^*$ is satisfied for $\forall n \in \mathbf{S}_n'$, we have

$$J_n/\theta_n < (\sum_{i\in\mathbf{S}_n'} J_i/\theta_i)/(|\mathbf{S}_n'|-1) \tag{21}$$

by combining the Eq. (15) and Eq. (20). The optimal strategy can also be obtained for the client n as Eq. (17). □

We can see that only clients satisfying Eq. (16) will be allowed to participate in HFL model training. Thus, the incentive mechanism we designed plays an essential role in client selection. When there are large-scale clients in the HFL framework, the incentive mechanism selects the best-performing clients to participate in the training, which greatly reduces training costs, such as computation and communication costs. The results will be sent to the blockchain platform, publishing the results and assembling the selected clients. Since the amount of uploaded data is significantly reduced, *the communication efficiency of the future networks will be effectively improved*.

B. Optimal Strategy in Edge Server's Subgame

Given the clients' optimal strategies, the edge server determines the gain for clients by balancing clients' computation performance and cost. It requires solving the following optimization problem,

$$\mathbf{P2:} \quad \max_{R_l} u_l^S(R_l), \quad s.t. \quad R_l \geq 0. \tag{22}$$

According to the optimal strategy $x_n^*, n \in S_l$ provided by clients, the edge server l needs to determine the strategy R_l to maximize its utilities. Similar to Eq. (17), the optimal strategy x_n^* can be rewritten as

$$x_n^* = Y_n R_l, \quad Y_n = \frac{|S_l'|-1}{\theta_n \sum_{i\in S_l'} J_i/\theta_i}\left(1 - \frac{(|S_l'|-1)J_n/\theta_n}{\sum_{i\in S_l'} J_i/\theta_i}\right). \tag{23}$$

$$x_n^* = \begin{cases} 0, & R_{\mu(n)} < (J_n \sum_{i\in\mathbf{S}_n\backslash\{n\}} \theta_i x_i)/\theta_n, \\ \sqrt{\frac{R_{\mu(n)} \cdot \sum_{i\in\mathbf{S}_n\backslash\{n\}} \theta_i x_i}{\theta_n J_n}} - \frac{\sum_{i\in\mathbf{S}_n\backslash\{n\}} \theta_i x_i}{\theta_n}, & \text{otherwise.} \end{cases} \tag{15}$$

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

Therefore, it is clear that the edge server l's utility is independent of x_n^* and x_{-n}^*.

Theorem 3 (Existence and Uniqueness of NE for Edge Servers): There is a unique NE as the optimal strategy R_l^* in the edge server's subgame that maximizes the utility function since it is a strictly concave maximization problem.

The proof of Theorem 3 is similar to that of Theorem 1. The key part of this proof is to prove that edge servers' utility function is strictly concave. Since the edge server's utility $u_l^S(R_l)$ can be proved to be concave, we could derive its optimal strategy by setting the first-order derivative of the utility as zero. Therefore, the optimal strategy R_l^* can be denoted by

$$R_l^* = \beta_l - \frac{\alpha_l}{\sum_{n \in S_l'} \theta_n Y_n P}. \tag{24}$$

C. Optimal Strategy in Cloud's Subgame

The cloud's behaviors are analyzed in this section, aiming to explore the cloud's optimal strategy for maximizing its utility. It requires us to solve the following optimization problem,

$$\textbf{P3:} \quad \max_P u^C(P), \quad s.t. \quad P \geq 0. \tag{25}$$

As the leader of the entire game, the cloud knows that there exits NE in clients and edge servers, so it only needs to maximize its own utility by finding its optimal strategy P^*.

Theorem 4 (Existence and Uniqueness of NE for Cloud): There exists a unique and stable NE in the cloud's subgame, that maximizes the cloud's utility.

The Proof of Theorem 4 is similar to those of Theorem 1 or Theorem 3. The critical part is to prove that the utility is a concave function. From this theorem, we can conclude that a unique NE strategy exists in the cloud's subgame. The cloud does not have an intention to change its decision under the NE. According to the concave property of the function $u^C(P)$, we can derive the optimal strategy of the cloud P^* by combining (17) and (24) to solve the following equation,

$$\partial u^C(P)/\partial P = 0. \tag{26}$$

Let $x_n^k,\ R_l^k\ P^k$ be the strategies of client n, edge server l and the cloud in the k-th step of the game process, respectively. The game process of the incentive mechanism in one round of HFL training is shown in Algorithm 1. Under the complete information, InFEDge in practice only adds a round of information exchange with little impact on communication costs. The computational cost contains the calculation of optimal strategy and the sorting operation.

D. HFL Process With Incentive Mechanism and Blockchain

The process of HFL with incentives empowered by the blockchain is shown in Fig. 8. First, the participants reach an agreement before performing each round of the HFL task. Second, participants perform training or aggregation according to the strategies given by the incentive mechanism. Third, the cloud earns the gain from evaluating the global model performance. The cloud and edge servers pay out the gain for the participants in the next layer based on edge servers' and clients' realized performances, respectively. Note that mobile devices participate in collaborative learning in most FL scenarios when they are in static conditions, such as in the battery-charging state. Hence, we assume that in the HFL framework, clients remain stable in each round of the game process, during which their geographical locations, computing capacity and other information keep almost unchanged.

Algorithm 1 The Game Process of InFEDge in Each Round

1: Initial all the strategies as $x_n^\kappa,\ R_l^\kappa\ P^\kappa, \kappa = 0$ and an extremely small value ι.
2: **while** $\forall\ n \in \mathcal{N}$, $|x_n^{\kappa+1} - x_n^\kappa| > \iota$ **do**
3: Client n adjusts x_n^κ (via Theorem 2), which is sent to the edge server and passed from the edge server to cloud.
4: Cloud adjusts P^κ via (26) and sends it to edge servers.
5: Edge server l adjusts R_l^κ via (24) and sends it to clients.
6: $\kappa = \kappa + 1$.
7: **end while**
8: Results finally obtained are the equilibrium solutions.

All game results among participants are implemented by predefined smart contracts in the form of transactions on the blockchain. The participants' identities and related data (e.g., multi-dimensional individual properties) cannot be revealed due to the transactional privacy in the blockchain. Moreover, the whole process and relevant key information are recorded on the chain, which avoids the risk of being lost and falsified and makes it easy for supervision. In the blockchain network, all the participants are peer nodes which equally restricted by the smart contracts, without the possibility of violation.

After finishing a round of the HFL task, the client's data quality can be updated in time as each client can measure the local accuracy of the model and calculate the value of data quality. The updated data quality will participate in the game process of the next round. In this paper, we decompose the HFL process into a recursive loop consisting of multiple rounds. The collaboration is performed for a specific time period recursively. Therefore, we will focus on analyzing the incentive mechanism in one round.

V. Incentive Mechanism Under Incomplete Information

There exist the following incomplete information issues for the participants in the HFL framework to do so [62]. **1)** A higher layer does not know which participants in the lower layer would like to join the model training due to the lack of prior knowledge. **2)** The individual properties, e.g., the local data quality and participating willingness of the lower layer, are unknown to the higher layer. **3)** The higher layer does not know the amount of available computation, communication resources or the data sizes used for model training of the lower layer. As a result, there may be suffering from too much cost when providing incentives to the lower layer. Hence, it is essential to *design an efficient incentive mechanism for reducing the impact of multi-layer incomplete information in the multi-layer framework*.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

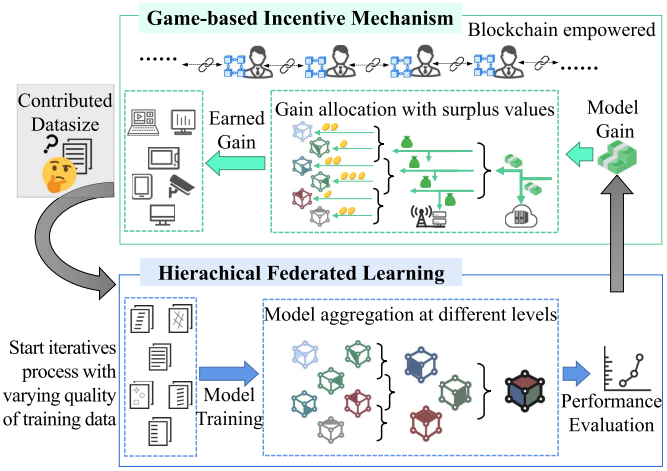


Fig. 8. An illustration of incentive mechanism for HFL in end-edge-cloud orchestrated systems. The entire system can be executed in an iterative manner, which helps to estimate system parameters (e.g., data quality).

The contract game (theory) as an efficient and powerful incentive mechanism is employed. However, most existing studies have studied the relationship between two-layer participants [45], [62], [63] and have not considered the complex case among multi-layers of participants. In this section, we propose *the hierarchical contract game* to capture interactions among participants and resolve their conflicting objectives under incomplete information. **The interactions in the cloud-edge layer and the interactions in the edge-client layer are modeled as two optimal contract problems, respectively**. We first introduce the contract game between edge servers and clients in the edge-client layer. The edge server l sets a contract for all types of clients, and the contract includes a series of data contributions and corresponding gain pairs as contract items. Clients are free to select a contract item. The objective of the edge server l is to maximize its utility by offering the optimal contract $\gamma_l = \{(x_n^*, x_{-n}^*, R_l^*), \forall n \in \mathcal{K}\}$.

A. Optimal Contract Design in the Edge-Client Layer

Conditions for Contract Feasibility: A feasible contract can attract clients to participate in the HFL task and consume all the gain provided by the edge server l. Moreover, it needs to ensure that each client only selects the contract item designed for its type. Particularly, the following individual rationality and incentive compatibility constraints need to be guaranteed.

Definition 2 (Individual Rationality (IR)): Each client only chooses to take on its contract item if its utility is non-negative, i.e., $\mathbb{E}[u_n^E(x_n, x_{-n}, R_l)] \geq 0$, specifically,

$$\mathbb{E}[f(x_n, x_{-n}) - J_n x_n] \geq 0.$$

Definition 3 (Incentive Compatible (IC)): The client must prefer to choose the contract item designed specifically for its own type instead of any other contract items j, i.e., $\mathbb{E}[u_n^E(x_n, x_{-n}, R_l) \geq u_n^E(x_j, x_{-n}, R_l)]$, specifically,

$$\mathbb{E}[f(x_n, x_{-n}) - J_n x_n] \geq \mathbb{E}[f(x_j, x_{-n}) - J_n x_j],$$

where the details can be expressed as

$$f(x_n, x_{-n}) = \theta_n x_n / (\theta_n x_n + \sum_{i \in S_l \setminus \{n\}} \theta_i x_i) R_l,$$
$$f(x_j, x_{-n}) = \theta_j x_j / (\theta_j x_j + \sum_{i \in S_l \setminus \{n\}} \theta_i x_i) R_l.$$

The IR constraints provide the client's necessary incentives to sign the contract. The IC constraints require that the client only maximize its utility by selecting the contract item that is designed for its own type. Thus, the type of each client is revealed to the edge server, which is called 'self-reveal'. If the contract item satisfies the IC and IR constraints, the contract item is feasible. Following the contract theory, the edge server aims to maximize its utility subject to the IR and IC constraints. Therefore, the optimal contract is the solution to the following optimization problem:

$$\begin{aligned} \max_{(x_n, x_{-n}, R_l)} \ & \mathbb{E}[u_l^S(x_n, x_{-n}, R_l)], \quad \forall n \in \mathcal{K}, \\ s.t. \ & \mathbb{E}[f(x_n, x_{-n}) - J_n x_n] \geq 0, \\ & \mathbb{E}[f(x_n, x_{-n}) - J_n x_n] \geq \mathbb{E}[f(x_j, x_{-n}) - J_n x_j], \\ & \quad n \neq j, \\ & 0 \leq x_n \leq |\mathcal{X}_n|, \\ & R_l \geq 0. \end{aligned} \tag{27}$$

Since multiple IR and IC constraints are non-convex, the problem given in (27) is not easy to solve directly. As such, we simplify these constraints by following a standard method in contract theory. In particular, we first simplify the IR constraints based on the following Lemma 1.

Lemma 1 (To Simplify the IR Constraints): If $\sigma_1 < \ldots < \sigma_K$, 1) under the IC constraints, we conclude that $u_n^E(x_n, x_{-n}, R_l) \geq \mathbb{E}[u_1^E(x_1, x_{-1}, R_l)]$ and thus IR constraints become $\mathbb{E}[u_1^E(x_1, x_{-1}, R_l)] \geq 0$; 2) in the optimal contract, under the IC constraints, the IR constraints for the lowest type σ_1 is binding, i.e., $\mathbb{E}[u_1^E(x_1, x_{-1}, R_l)] = 0$.

From Lemma 1, we conclude that if the lowest client type among all the IR constraints binds, then the other types will automatically hold under IC constraints. Next, the IC constraints can be simplified by the following lemmas, whose proofs are omitted for brevity.

Definition 4 (Monotonicity): 1) Under the IC constraints, if $\sigma_n < \sigma_j$, then $x_n < x_j$.

2) If the contract satisfies IC constraints, then the monotonicity constraint holds, i.e., $x_n \leq x_j$ if $\sigma_n \leq \sigma_j$.

3) Under the IC constraints, if $\sigma_n < \sigma_j$, then we can get $f(x_n, x_{-n}) < f(x_j, x_{-n})$.

Definition 4 implies that the utility should be higher for higher type-j clients. Otherwise, clients of all types would like to choose higher utility with lower cost and data quality.

Definition 5 (Simplify the IC Constraints): With the monotonicity constraints, the IC constraints can be reduced to the following two constraints:

1) The Local Upward Incentive Constraint (LUIC):

$$\mathbb{E}[u_n^E(x_n, x_{-n}, R_l)] \geq \mathbb{E}[u_n^E(x_{i+1}, x_{-n}, R_l)].$$

2) The Local Downward Incentive Constraint (LDIC):

$$\mathbb{E}[u_n^E(x_n, x_{-n}, R_{l,i})] \geq \mathbb{E}[u_n^E(x_{n-1}, x_{-n}, R_l)].$$

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

LDIC and LUIC are sufficient conditions for the IC constraints and they can replace the IC constraints to reduce the space of constraints. Definition 6 is employed to further reduce the LDIC and LUIC constraints.

Definition 6 (Further Simplify the IC Constraints): In the optimal contract, the IC constraints can be replaced by $\mathbb{E}[u_n^E(x_n, x_{-n}, R_l)] = \mathbb{E}[u_n^E(x_{n+1}, x_{-n}, R_l)]$.

By using the simplified constraints given above, the optimization problem can be transformed to:

$$\max_{(x_n, x_{-n}, R_l)} \mathbb{E}[u_l^S(x_n, x_{-n}, R_l)], \quad \forall i \in \mathcal{K},$$

$$\begin{aligned} s.t. \quad & \mathbb{E}[f(x_1, x_{-1}) - J_n x_1] = 0, \\ & \mathbb{E}[f(x_n, x_{-n}) - J_n x_n] \\ & = \mathbb{E}[f(x_{i+1}, x_{-i}) - J_n x_{i+1}], \quad i \neq j, \\ & x_1 < x_2 < \ldots < x_K, \\ & 0 \leq x_n \leq |\mathcal{X}_n|, \\ & \sigma_n \geq 0, R_l \geq 0. \end{aligned} \tag{28}$$

B. Optimal Contract Solution in the Edge-Client Layer

The optimization problem above can be solved to obtain the optimal contract as follows: firstly, a standard method in the contract game is applied to solve the relaxed problem without the monotonicity constraint. Then, the solution is verified to guarantee the monotonicity constraint. Specifically, by iterating the first and second constraints in (28), the following equation can be derived: $x_n = x_1 - \sum_{i=1}^{n-1} w_i$ where $w_i = \sigma_i(f(x_n, x_{-n}) - f(x_{n+1}, x_{-n}))$. Meanwhile, the following equation holds:

$$R_l = \sum_{n \in S_l} \theta_n x_n / \sigma_1. \tag{29}$$

Substituting Eq. (29) into the objective function of the edge server, we can remove the dependence of $\mathbb{E}[u_l^S(x_n, x_{-n}, R_l)]$ on R_l and rewrite $h(R_l)$ as

$$\begin{aligned} h'(x_n, x_{-n}) = \ln(\alpha_l + P \sum_{n=1}^{K} x_1 \theta_n - P \sum_{n=1}^{K} T_n f(x_n, x_{-n}) \\ + P \sum_{n=1}^{K} T_n f(x_{n+1}, x_{-n})), \end{aligned} \tag{30}$$

where $T_n = \sigma_n \sum_{i=n+1}^{K} \theta_n$. Accordingly, the problem in (28) can be expressed as

$$\begin{aligned} \max_{(x_n, x_{-n})} \quad & \mathbb{E}[u_l^S(x_n, x_{-n})] \\ & = \mathbb{E}[h'(x_n, x_{-n}) - cost_l^S(\frac{\sum_{n \in S_l} \theta_n x_n}{\sigma_1})], \\ s.t. \quad & 0 \leq x_n \leq |\mathcal{X}_n|, \\ & \sigma_n \geq 0, \quad \forall n \in \mathcal{K}. \end{aligned} \tag{31}$$

Taking the second-order partial derivative of the objective function given in Eq. (31) with respect to x_n, we can easily conclude that the second-order partial derivative is less than 0. Thus, it is a concave function with respect to x_n. Taking the first-order partial derivative of Eq. (31) with respect to x_n, we obtain $\partial \mathbb{E}[u^S(x_n, x_{-n})]/\partial x_n$. According to Fermat's Theorem, x_n^* in the optimal contract is derived from solving $\partial \mathbb{E}[u_l^S(x_n, x_{-n})]/\partial x_n = 0$. R_l^* can be solved via Eq. (29) after x_n^* is obtained. Until now, we have achieved the optimal contract that maximizes the utility of the edge server l in the edge-end layer while satisfying the constraints of IR and IC. The proposed method alleviates the anxiety caused by incomplete information between edge servers and clients. Because if a private blockchain is used, the completed information is not visible to each participant.

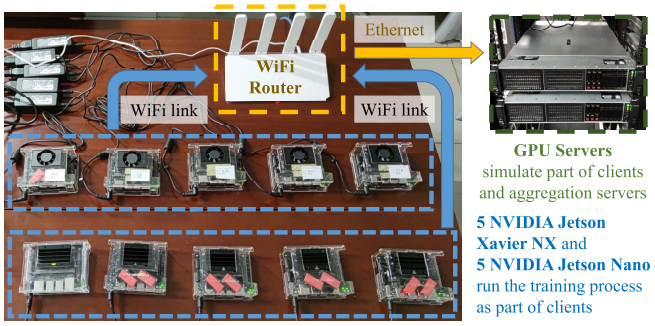


Fig. 9. The HFL proof-of-concept testbed for experimental evaluations.

C. Optimal Contract Problem in the Cloud-Edge Layer

The interactions among the cloud and edge servers can be similarly analyzed as the above. Due to space limits, we omit details of interactions among the cloud and edge servers. The corresponding optimal contract is the solution to the following optimization problem.

$$\begin{aligned} \max_{(R_l, R_{-l})} \quad & \mathbb{E}\{\lambda \cdot g[g_2(R_l, R_{-l})] - \tau_c g_2(R_l, R_{-l}) P\}, \quad \forall l \in \mathcal{Q}, \\ s.t. \quad & g_2(R_l, R_{-l}) = \sum_{i \in \mathcal{Q}} (\sigma_1 R_1 - \phi_i \psi_i(R_i) + \phi_i \psi_i(R_{i+1})), \\ & \phi_i = (Q - i)\sigma_1 \beta_i, \quad \psi_i(R_i) = \ln(\alpha_i + \sigma_1 R_i P), \\ & P = [\exp(cost_1^S(R_1)) - \alpha_1]/(\sigma_1 R_1), \quad R_l \geq 0. \end{aligned} \tag{32}$$

It is worth noting that the proposed hierarchical contract game problem for the incentive mechanism in the HFL is transformed into two optimal contract problems in the cloud-edge layer and in the edge-client layer, respectively. The first is to solve the optimal contract problem (32) in the cloud-edge layer. Further, the solutions in the cloud-edge layer are introduced into the edge-client layer for solving the optimal contract problem (31). The two optimal contract games proceed iteratively until all strategies in the system converge to a fixed value. Similarly, game results will be sent to the blockchain platform, where corresponding transactions will be executed.

VI. Experimental Evaluation

A. Experimental Setup

1) Testbed Overview: As shown in Fig. 9, we conduct the experiments on a proof-of-concept testbed. The router

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.



Fig. 10. The construction of blockchain with the management platform.

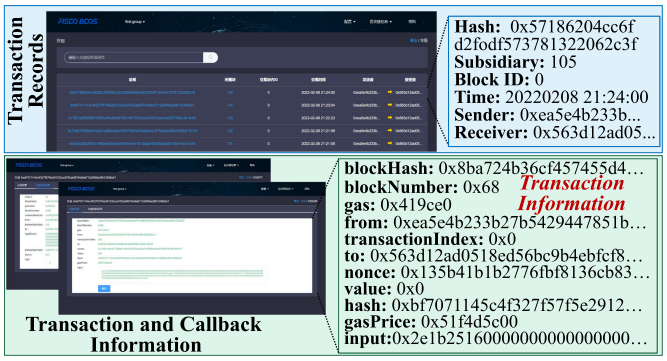


Fig. 11. We could interact with the consortium blockchain for acquiring corresponding information, such as the current block, transaction records, transaction and callback information, etc.

connects 10 embedded devices and 2 GPU servers via WiFi and Ethernet. We assume that the HFL system consists of 1 cloud, 5 edge servers, and up to 100 clients in the experiments. Some of the clients in the experiment are running on real embedded devices, e.g. NVIDIA Jetson Xavier NX and NVIDIA Jetson Nano. We leverage GPU servers with multi-thread programming to simulate aggregation operations and the training process of the rest clients. The training and communication process is implemented with PyTorch and the web micro-framework Flask.[1]

We refer to [40] for the numerical setting of computing and communication parameters and randomly assign simulated clients into two types: high and low computing power devices where the computing power of the former is twice that of the latter by default. We assume this with two primary considerations: a) The image processing rate of the Xavier NX is significantly better than that of the Nano in official benchmarks,[2] and b) Ensuring that the training cost in GPU servers is in line with the training cost of embedded devices.

2) Implementation of Smart Contract of Blockchain: To factually provide blockchain services for energy trading, we build a consortium blockchain based on FISCO BCOS, a stable and efficient blockchain underlying platform. As shown in Fig. 10, the devices for implementing a four-node blockchain network are I) A laptop with 16GB RAM and Intel i7-11800H CPU. II) A virtual machine with 8GB RAM and 30GB SCSI. III) A router providing wireless access.

A smart contract is designed to realize the automatic execution of account trading and clearing. In order to provide better transaction services and more convenient management, a common set of components are built between the incentive mechanism in HFL and the consortium blockchain nodes based on WeBASE, a comprehensive middleware platform assisting in developing blockchain-based distributed applications. It provides unified management of chains, contracts, private keys, and applications. Further, the average transaction cost (i.e., the gas value), the maximum throughput, and latency for the execution of the smart contracts are 4,300,000, 150 MB/s, and 0.8 ms-4 ms, respectively. We adopt the '4 hosts, single chain and 4 nodes' (satisfying the number of nodes per cluster consensus is at least $(3 \times n + 1)$ when multiple institutions form a cluster, with n representing the number of fault-tolerant nodes in the system as an integer greater than or equal to 1) as the basic configuration. Furthermore, the state of the blockchain network, e.g., average block size, blockchain size and average block time are 1480 Kb, '4 hosts, single chain and 4 nodes with block height 105' and 750 ms, respectively.

To provide distributed blockchain service, we open the node front function for each node and compile and deploy the smart contract on the constructed blockchain using the built management platform. Additionally, after testing the smart contract, strict identity authentication identities and calling modes are set to prevent malicious attacks caused by permission problems. Besides, we could interact with the consortium blockchain for information, such as the current block, transaction records, callback information and other details, as illustrated in Fig. 11.

3) Dataset Preprocessing: We consider the following three well-known datasets for the image classification of 10 categories as the representative AI applications of HFL: a) **MNIST** is a handwritten digital dataset in which each sample is a 28×28 grayscale image. b) **Fashion-MNIST** is a cloth dataset with samples in the same format as MNIST. c) **CIFAR-10** is a universal object dataset where each sample is a 32×32 RGB image. We mainly demonstrate the experimental results on MNIST and obtain similar results on other datasets. Each of the above three datasets has at least 50,000 samples, which is sufficient for evaluation requirements.

We consider clients' heterogeneity in data quality and divide them into three equal proportions: low, medium, and high. These three types of clients have 4, 5, and 6 categories of data, respectively. For each category one client has, we randomly select 20% samples from all the samples labelled with the category to create the client's dataset. Note that this percentage is reasonably chosen to avoid too many training samples under a single edge server.

[1]https://flask.palletsprojects.com/en/2.0.x/
[2]https://developer.nvidia.com/embedded/jetson-benchmarks

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

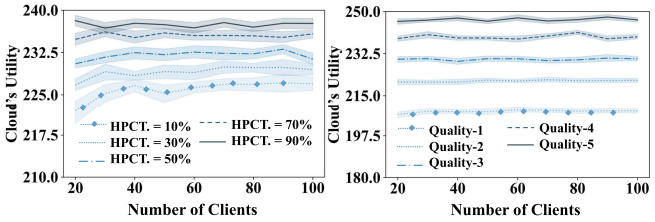


Fig. 12. The cloud's utility versus the total number of clients (under different (a) the percentage of high quality clients (HPCT.) and (b) the data quality).

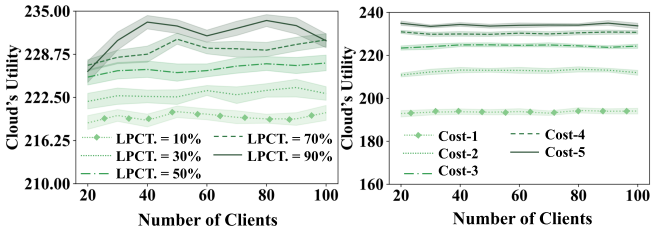


Fig. 13. The cloud's utility versus the total number of clients (under different (a) the percentage of low cost clients (LPCT.) and (b) the training cost).

4) Training Parameter: For the evaluation of MNIST and Fashion-MNIST, we adopt a fully connected network with two hidden layers (256, 64 units, respectively) and train 200 rounds for each client. Considering the difficulty of training on CIFAR-10, we adopt ResNet-14 as the training model and train 1000 rounds for each client. We use the ADAM optimizer with a learning rate of 10^{-3} and set the batch size to 32. We assume that each client uploads its model parameter to the edge server per 2 rounds. Each edge server uploads its parameter per 6 rounds. The whole training process in each experiment is executed in a synchronous setting, i.e., each aggregation of an edge server starts after all connected clients upload their model parameters, and the aggregation on the cloud is performed similarly.

B. Impact of Multi-Dimensional Individual Properties

We evaluate the impact of the percentage of high-quality clients and clients' data quality on the cloud's utility. For the former, we changed the proportion of high-quality clients from 10% to 90% and ensured that medium-quality clients had the same proportion as low-quality clients. For the latter, we designed five different sets of data quality (from Quality-1 to Quality-5), and the number of data categories owned by low, medium and high quality clients are (2, 3, 4), (3, 4, 5), (4, 5, 6), (5, 6, 7), and (6, 7, 8) respectively.

As shown in Fig. 12, the increasing speed of the cloud's utility tends to slow down, i.e., the influence of the number of clients on the cloud's utility is diminishing. For the case HPCT. = 10%, its increase in utility is about 75%, 250%, 650% and 840% higher than the others, respectively. In a system with its scale increases, the cloud's utility will gradually become stable rather than continue to grow. Further, in the case of the low percentage of the high-quality client (shown in Fig. 12(a)), the upward trend is more pronounced since few high-quality clients result in low utility. Additionally, these

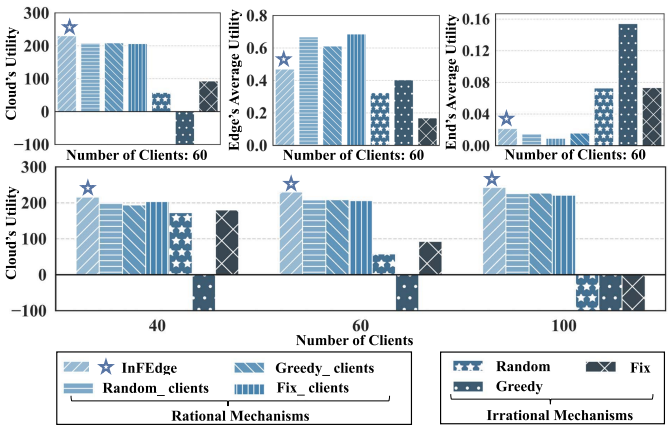


Fig. 14. Comparison of (a) cloud's utility (b) edge server's average utility and (c) client's average utility under different baselines. (d) The cloud's utility versus the number of clients under different baselines.

can also be shown in the indicator of variance. The results in Fig. 12(b) show the increase of the cloud's utility with the improvement of data quality. Meanwhile, we also conduct similar experiments to analyze the impact of another dimension of property, training cost. We consider five sets of training costs (from Cost-5 to Cost-1, training costs gradually increases). Both experimental results demonstrate that InFEDge is in line with the preferences of the cloud, as shown in Fig. 13.

C. Performance Comparison With Baselines

Fig. 14 compares InFEDge with some baseline mechanisms, including rational and irrational mechanisms. Rational mechanisms (Random-clients, Greedy-clients and Fixed-clients) select random clients, all clients, and half of the clients to participate in games and training and are only different from InFEDge's design in user selection. Irrational mechanisms (Random, Greedy, Fix) ask random clients to contribute random data, all the clients to contribute all the data, and half of the clients to contribute half of their data, respectively, without considering the selfishness and rationality of clients or edge servers.

The superiority of InFEDge over the irrational and rational mechanisms comes from following individual rationality and client selection, respectively. Interestingly, in addition to the edge server (27.5% less on average), InFEDge has a higher utility than other rational mechanisms in terms of cloud and clients (21.2% and 32.6% higher on average, respectively) (Fig. 14(a)-(c)). The cloud, edge servers and clients play roles similar to those of consumers, middlemen, and manufacturers in the markets. Consumers' and manufacturers' utilities can be increased by reducing middlemen's earned. Therefore, designing an effective incentive mechanism (making clients allocate more incentives to contribute more data and improve the cloud's utility) is achieved by enhancing clients' utilities and impacting edge servers' utility.

As shown in Table II, we evaluate InFEDge on different datasets. Compared to the other two datasets, the results trained on the CIFAR-10 dataset are less in terms of cloud's

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF DIFFERENT DATASETS FOR THE FOLLOWING ITEM

Item / Dataset	Cloud's utility	Number of participants	Average data contribution
MNIST	231.28	34	35.24%
Fashion-MNIST	231.24	33	35.92%
CIFAR-10	156.89	32	8.00%

utility and the amount of data contribution due to the difficulty of training a good model from the dataset.

D. Impact of Deviation From NE and the Performance Under Incomplete Information

As shown in Fig. 15, forcing the cloud's unit service price P to rise beyond the NE value only provides a slight increase in model performance. By contrast, it is much more sensitive to the cloud's strategy reductions ($\tan(A_1) < \tan(A_2)$, as shown in Fig. 15(a)). Therefore, we could not blindly increase P only considering the model's performance. InFEDge is an excellent way to achieve the balance between gain and cost. Furthermore, we present the feasibility of the incentive mechanism under incomplete information in Fig. 15(b). The IC constraint is validated because each client can achieve maximum utility only when selecting the contract items accurately. Moreover, the non-negative utility validates the IR constraints.

E. Performance Comparison Between Complete and Incomplete Information

As shown in Fig. 16, for a certain number of clients, the cloud's and edge servers' utilities under incomplete information are higher than those under complete information (2.65% and 5.13% higher on average, respectively). In contrast, the clients' result is the opposite (77% less on average). As leaders, the cloud and edge servers only provide limited contract items to the followers and extract more gain from them under incomplete information. Nevertheless, rational participants can optimize their utilities under complete information, resulting in utility reduction for leaders. Although the leaders need to consider the IR and IC constraints while designing the contract items under incomplete information, these constraints have a small impact on maximizing the utilities of the followers [64].

VII. USE CASE OF INFEDGE FOR 5G RESOURCE OPTIMIZATION

With the explosive growth of mobile devices, networks are under tremendous pressure of data transmission. It has become a hot issue to reduce communication delay and improve communication efficiency in 5G scenarios [65]. As illustrated in Fig. 17, the user allocation problem for 5G base stations (BSs) under the end-edge-cloud collaborative architecture has not been well solved. The greedy allocation method lacking adaptability (e.g., proximity principle) used in most cases may incur severe network congestion, generating more latency and negatively affecting the quality of service (QoS).

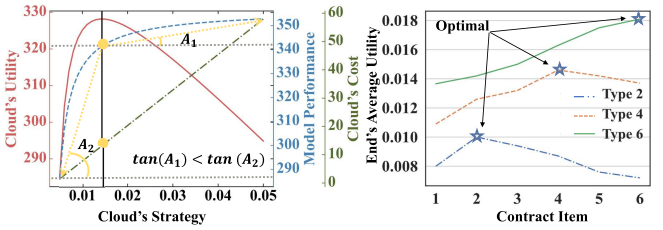


Fig. 15. (a) Dependence of the cloud's utility (red), the model performance (blue) and the cost (green) on strategy P where the equilibrium is calculated to be P^* (black line). (b) The average utility of client versus contract items under type 2, type 4 and type 6.

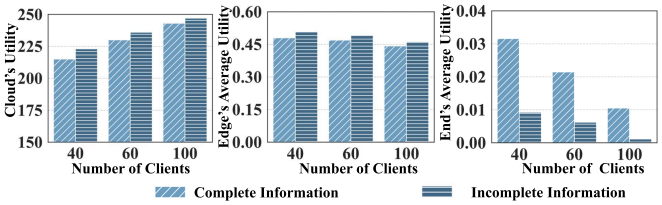


Fig. 16. (a) Cloud's utility (b) Edge server's average utility and (c) Client's average utility versus the number of clients under complete and incomplete information.

A. Problem Formulation

The end-edge-cloud collaborative architecture in which the user allocation problem occurs is similar to that in Section III-A. There are a cloud, some 5G macro BSs, 5G micro BSs and U 5G users (denoted by $\mathcal{U}$) distributed in the system. Each user u has the data loads D_u and two-dimensional coordinates o_u^X, o_u^Y. In each time slot, users will be allocated to the designated 5G BSs, which serve multiple users simultaneously. If BS b serves user u, then it can be denoted as $E_{b,u} = 1$, otherwise $E_{b,u} = 0$. We assume that the system will not proceed to the next round until all data have been transmitted.

B. The Communications Model

The communications between users and BS are established through the cellular network. We assume that users connected to the same BS have the same communication rate without considering the shadow effect and Signal to Interference plus Noise Ratio. Further, since the realistic network resources are inversely proportional to the Euclidean distance $l_{b,u}$ between u and b, realistic network resources available to user u is $r_{b,u}^f = k \cdot \frac{R_b}{N_b l_{b,u}}$, where k is a constant parameter. R_b is network resources from b, N_b is the number of users connected to b. According to the Shannon-Hartley theorem, the peak rate $r_{b,u}$ between u and b can be described as

$$r_{b,u} = G \cdot r_{b,u}^f \log_2\left(1 + \frac{\sum_{i \in \mathcal{B}, i \neq b} r_{i,u}^f}{r_{b,u}^f + v^2}\right), \tag{33}$$

where G is the fraction of bandwidth and v^2 is the Gaussian noise in the 5G networks.

Note that we only consider the download process, as the uploaded data and delay are much smaller than the

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

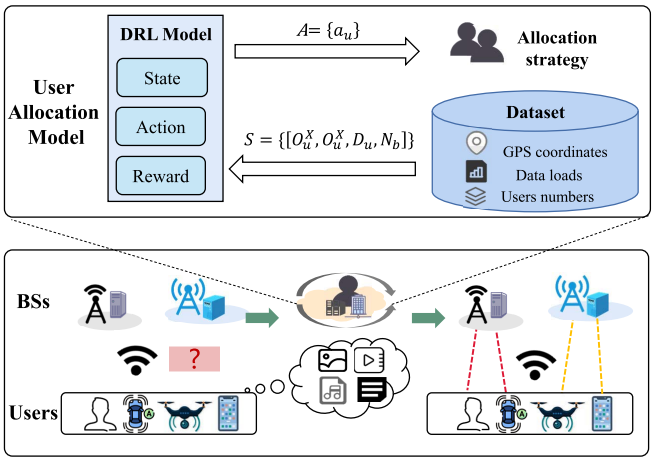


Fig. 17. The problem of user allocation for 5G BSs under the end-edge-cloud collaborative architecture is taken as our use case application.

downloaded ones. Hence, the download delay of BS b is $t_b = \sum_{u\in\mathcal{U}} t_{b,u}$, where $t_{b,u} = D_u/r_{b,u}$. Finally, the entire delay of the system equals $T = \min(\max_{b\in\mathcal{B}}(\sum_{u\in\mathcal{U}} t_{b,u}), T_0)$, in which T_0 represents the maximum threshold of delay defined by different applications. A synchronization mechanism is used in the user allocation scenario and the delay depends on the longest delay in the whole system. Since a longer delay will further deteriorate users' QoS, the optimization objective can be expressed as

$$\min_{\{E_{b,u}, b\in\mathcal{B}, u\in\mathcal{U}\}} T = \min_{\{E_{b,u}, b\in\mathcal{B}, u\in\mathcal{U}\}} [\min(\max_{b\in\mathcal{B}}(\sum_{u\in\mathcal{U}} t_{b,u}), T_0)]. \tag{34}$$

C. DRL Algorithm

In order to tackle the optimization problem above, an adaptive user allocation method based on Deep Reinforcement Learning (DRL) is proposed to optimize the communications and networking environment and reduce network delay (Fig. 17). The corresponding state S, action A and reward R of the DRL method are defined as follows.

- **State** S: $S = \{s_u = [o_u^X, o_u^Y, D_u, N_b], u \in \mathcal{U}\}$ where characters in $[o_u^X, o_u^Y, D_u, N_b]$ stand for x-coordinate, y-coordinate, the data loads of user u and the total number of users to be processed together, respectively.
- **Action** A: $A = \{a_u \in \{0, 1, \ldots, C\}, u \in \mathcal{U}\}$. a_u is the allocation strategy, which indicates that user u selects the a_u closest BS to receive data. C stands for the maximum number of BSs for each user to select.
- **Reward** R: After the current state transfers into the next state, the system will gain the reward R, which equals the negative of the system delay in a time slot, formally:

$$R = -T = -\min(\max_{b\in\mathcal{B}}(\sum_{u\in\mathcal{U}} t_{b,u}), T_0)\}. \tag{35}$$

Based on *a real-world 5G users and BSs dataset*, the DRL model is performed distributed training under the end-edge-cloud collaborative architecture. In practice, users will send their basic information (i.e., GPS coordinates and data loads) into the well-trained DRL model, which will generate the corresponding allocation strategies to establish data connections between 5G users and BSs.

D. The Necessity of Applying HFL and InFEDge

Considering common network issues, such as network congestion, data privacy, etc., it is not feasible to train the model in the cloud. A practical solution for implementing distributed training is to store the data at BSs and use the FL framework. Further, massive micro BSs are geographically scattered at the edge of the network and connect to the remote cloud via long-distance transmission, incurring high communication costs, severe network congestion and training delay. Therefore, *it is a feasible and effective method to apply the HFL framework to train the DRL model collaboratively*.

(1) **5G micro BSs (clients):** They collect and store large amounts of user allocation data to train the DRL model.
(2) **5G macro BSs (edge servers):** Being closer to the cloud than micro BSs, they benefit from higher bandwidth or computing power and conduct edge aggregations.
(3) **The remote cloud (cloud):** The cloud publishes the training task and conducts cloud aggregations.

However, it is an open question whether the HFL framework can be deployed successfully in practice. BSs training is a distributed DRL training that wastes a lot of energy in data collection and consumes various resources, such as power, bandwidth, and computing resources. As a result, few BSs devote themselves to the DRL model training. *Providing compensation and economic incentives for BSs is an effective way to promote cooperation among them in the model training.*

(1) **Multi-dimensional Individual Properties.** The varying data quality collected by micro 5G BSs and different communication and computing resources available for model training make it challenging to offer fair economic incentives. In addition, macro 5G BSs may belong to different companies with different willingness to participate.
(2) **Incomplete Information.** Since 5G BSs may belong to different companies, private information is not disclosed to the public. Therefore, there exists an incomplete information issue in designing the incentive mechanism.
(3) **Unreliable participants.** Some BSs deviate from the learning protocol and send corrupt updates to poison the global model. We lack a fair and transparent system to provide economic incentives and ensure data privacy.

All of the above challenges in the realistic user allocation scenario for 5G BSs fit perfectly with those that the proposed InFEDge solves. The three most important elements of InFEDge, i.e., HFL, the incentive mechanism and the blockchain, deal with the training, practical deployment and other aspects encountered in solving the 5G user allocation problem, respectively. Therefore, as shown in Fig. 18, it is evident that the proposed blockchain-based incentive mechanism in the HFL, InFEDge, is an appropriate solution to deal with the user allocation problem.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

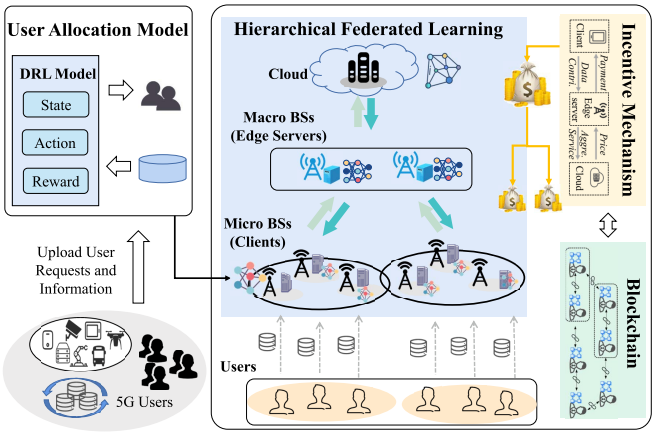


Fig. 18. The user allocation problem for 5G BSs fits perfectly with those InFEDge solved. InFEDge is the best solution to user allocation issues.

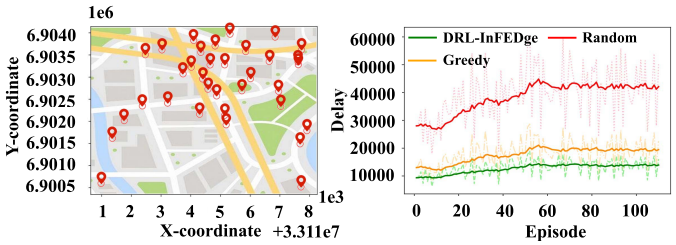


Fig. 19. (a) The coordinate data of BSs. (b) Effectiveness of the InFEDge for solving the 5G user allocation problem for 5G BSs under the end-edge-cloud collaborative architecture.

E. Performance for Solving the User Allocation Problem

To show the performance and superiority of our work in the real-world use case,[3] we compare it with the Random method and the Greedy method. In the Random algorithm, BSs act randomly, while BSs select the best action only based on the current state in the greedy algorithm, without considering the impact on historical or future states. As shown in Fig. 19, in the early stage of training, the time derived with the InFEDge method is 26.2% and 65.8% on average less than the Greedy and Random methods, respectively. Moreover, in the later stage of training, the results even reached 27.7% and 66.2% on average. The result confirmed that InFEDge is effective and plays well in the 5G user allocation problem for 5G BSs under the end-edge-cloud collaborative architecture.

VIII. Conclusion

In this article, we investigate the issue of incentive mechanism design for the HFL under the end-edge-cloud collaborative architecture. We propose InFEDge, a blockchain-based incentive mechanism, to address the challenges of multi-dimensional individual properties, incomplete information, and unreliable participants. In particular, we prove the existence and uniqueness of the Nash equilibrium with the closed-form solution in the presence of multi-dimensional individual properties. Then, we consider the incentive issue under the incomplete information of multi-dimensional individual properties and obtain the optimal solution. Moreover, implementation is based on blockchain, which prevents unreliable participants' disturbance, further ensuring privacy and providing a credible and transparent environment. Experimental results present the superiority of InFEDge compared with baseline schemes and show the practicability used to solve real-world problems.

[3]The real-world data is collected in https://anonymous.4open.science/r/The-5G-BSs-and-Users-Dataset-315C

References

[1] J. Wu, S. Guo, H. Huang, W. Liu, and Y. Xiang, "Information and communications technologies for sustainable development goals: State-of-the-art, needs and perspectives," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 3, pp. 2389–2406, 3rd Quart., 2018.

[2] M. Chiang and T. Zhang, "Fog and IoT: An overview of research opportunities," *IEEE Internet Things J.*, vol. 3, no. 6, pp. 854–864, Dec. 2016.

[3] P. Zhang, C. Wang, C. Jiang, and Z. Han, "Deep reinforcement learning assisted federated learning algorithm for data management of IIoT," *IEEE Trans. Ind. Informat.*, vol. 17, no. 12, pp. 8475–8484, Dec. 2021.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: http://www.deeplearningbook.org

[5] H. Ding, Y. Guo, X. Li, and Y. Fang, "Beef up the edge: Spectrum-aware placement of edge computing services for the Internet of Things," *IEEE Trans. Mobile Comput.*, vol. 18, no. 12, pp. 2783–2795, Dec. 2019.

[6] Z. Xiong, J. Kang, D. Niyato, P. Wang, and H. V. Poor, "Cloud/edge computing service management in blockchain networks: Multi-leader multi-follower game-based ADMM for pricing," *IEEE Trans. Serv. Comput.*, vol. 13, no. 2, pp. 356–367, Mar./Apr. 2020.

[7] Z. Yang, B. Liang, and W. Ji, "An intelligent end–edge–cloud architecture for visual IoT-assisted healthcare systems," *IEEE Internet Things J.*, vol. 8, no. 23, pp. 16779–16786, Dec. 2021.

[8] W. Y. B. Lim et al., "Hierarchical incentive mechanism design for federated machine learning in mobile networks," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9575–9588, Oct. 2020.

[9] J. Wu et al., "Hierarchical personalized federated learning for user modeling," in *Proc. WWW*, 2021, pp. 957–968.

[10] J. Wang, S. Wang, R. Chen, and M. Ji, "Local averaging helps: Hierarchical federated learning and convergence analysis," 2020, *arXiv:2010.12998*.

[11] M. S. H. Abad, E. Ozfatura, D. Gunduz, and O. Ercetin, "Hierarchical federated learning ACROSS heterogeneous cellular networks," in *Proc. ICASSP*, May 2020, pp. 8866–8870.

[12] H. Zheng, M. Gao, Z. Chen, and X. Feng, "A distributed hierarchical deep computation model for federated learning in edge computing," *IEEE Trans. Ind. Informat.*, vol. 17, no. 12, pp. 7946–7956, Dec. 2021.

[13] L. U. Khan et al., "Federated learning for edge networks: Resource optimization and incentive mechanism," *IEEE Commun. Mag.*, vol. 58, no. 10, pp. 88–93, Oct. 2020.

[14] L. Liu, J. Zhang, S. H. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *Proc. IEEE ICC*, Jun. 2020, pp. 1–6.

[15] S. Wang et al., "When edge meets learning: Adaptive control for resource-constrained distributed machine learning," in *Proc. IEEE INFOCOM*, Apr. 2018, pp. 63–71.

[16] Z. M. Fadlullah and N. Kato, "On smart IoT remote sensing over integrated terrestrial-aerial-space networks: An asynchronous federated learning approach," *IEEE Netw.*, vol. 35, no. 5, pp. 129–135, Sep. 2021.

[17] W. Y. B. Lim et al., "Incentive mechanism design for resource sharing in collaborative edge learning," *CoRR*, vol. abs/2006.00511, pp. 1–7, May 2020.

[18] C. A. Holt and S. K. Laury, "Risk aversion and incentive effects," *Amer. Econ. Rev.*, vol. 92, no. 5, pp. 1644–1655, Nov. 2002.

[19] N. Ding, Z. Fang, and J. Huang, "Optimal contract design for efficient federated learning with multi-dimensional private information," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 186–200, Jan. 2021.

[20] X. Lin, J. Wu, J. Li, X. Zheng, and G. Li, "Friend-as-learner: Socially-driven trustworthy and efficient wireless federated edge learning," *IEEE Trans. Mobile Comput.*, early access, Apr. 21, 2021, doi: 10.1109/TMC.2021.3074816.

[21] P. Zhang, Y. Hong, N. Kumar, M. Alazab, M. D. Alshehri, and C. Jiang, "BC-EdgeFL: A defensive transmission model based on blockchain-assisted reinforced federated learning in IIoT environment," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3551–3561, May 2022.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.

[22] M. S. Ali, M. Vecchio, M. Pincheira, K. Dolui, F. Antonelli, and M. H. Rehmani, "Applications of blockchains in the Internet of Things: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1676–1717, 2nd Quart., 2019.

[23] Y. Li, C. Chen, N. Liu, H. Huang, Z. Zheng, and Q. Yan, "A blockchain-based decentralized federated learning framework with committee consensus," *IEEE Netw.*, vol. 35, no. 1, pp. 234–241, Jan. 2021.

[24] A. Dorri, S. S. Kanhere, R. Jurdak, and P. Gauravaram, "Blockchain for IoT security and privacy: The case study of a smart home," in *Proc. IEEE PerCom Workshops*, Mar. 2017, pp. 618–623.

[25] E. J. De Aguiar, B. S. Faiçal, B. Krishnamachari, and J. Ueyama, "A survey of blockchain-based strategies for healthcare," *ACM Comput. Surv.*, vol. 53, no. 2, pp. 1–27, Mar. 2021.

[26] E. Bertino, A. Kundu, and Z. Sura, "Data transparency with blockchain and AI ethics," *J. Data Inf. Qual.*, vol. 11, no. 4, pp. 1–8, Sep. 2019.

[27] H. Wu, A. Ashikhmin, X. Wang, C. Li, S. Yang, and L. Zhang, "Distributed error correction coding scheme for low storage blockchain systems," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7054–7071, Aug. 2020.

[28] L. Cui, S. Yang, Z. Chen, Y. Pan, M. Xu, and K. Xu, "An efficient and compacted DAG-based blockchain protocol for industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 16, no. 6, pp. 4134–4145, Jun. 2020.

[29] L. Wang, W. Wang, and B. Li, "CMFL: Mitigating communication overhead for federated learning," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Dallas, TX, USA, Jul. 2019, pp. 954–964.

[30] Y. Jin, L. Jiao, Z. Qian, S. Zhang, S. Lu, and X. Wang, "Resource-efficient and convergence-preserving online participant selection in federated learning," in *Proc. IEEE ICDCS*, Nov. 2020, pp. 606–616.

[31] C. Wang, Y. Yang, and P. Zhou, "Towards efficient scheduling of federated mobile devices under computational and statistical heterogeneity," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 394–410, Feb. 2021.

[32] J.-H. Ahn, O. Simeone, and J. Kang, "Wireless federated distillation for distributed edge learning with heterogeneous data," in *Proc. IEEE PIMRC*, Sep. 2019, pp. 1–6.

[33] Y. Jiang, S. Wang, B. J. Ko, W. Lee, and L. Tassiulas, "Model pruning enables efficient federated learning on edge devices," *CoRR*, vol. abs/1909.12326, pp. 1–22, Sep. 2019.

[34] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. NIPS*, 2017, pp. 1709–1720.

[35] Q. Hu, F. Li, X. Zou, and Y. Xiao, "Correlated participation decision making for federated edge learning," in *Proc. GLOBECOM*, Dec. 2020, pp. 1–6.

[36] R. Saha, S. Misra, and P. K. Deb, "FogFL: Fog-assisted federated learning for resource-constrained IoT devices," *IEEE Internet Things J.*, vol. 8, no. 10, pp. 8456–8463, May 2021.

[37] J. Pang, Y. Huang, Z. Xie, Q. Han, and Z. Cai, "Realizing the heterogeneity: A self-organized federated learning framework for IoT," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3088–3098, Mar. 2021.

[38] M. Assran, J. Romoff, N. Ballas, J. Pineau, and M. Rabbat, "Gossip-based actor-learner architectures for deep reinforcement learning," in *Proc. NeurIPS*, 2019, pp. 1–11.

[39] O. Marfoq, C. Xu, G. Neglia, and R. Vidal, "Throughput-optimal topology design for cross-silo federated learning," in *Proc. NeurIPS*, 2020, pp. 19478–19487.

[40] S. Luo, X. Chen, Q. Wu, Z. Zhou, and S. Yu, "HFEL: Joint edge association and resource allocation for cost-efficient hierarchical federated edge learning," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6535–6548, Oct. 2020.

[41] Z. Chai, Y. Chen, L. Zhao, Y. Cheng, and H. Rangwala, "FedAT: A high-performance and communication-efficient federated learning system with asynchronous tiers," *CoRR*, vol. abs/2010.05958, pp. 1–16, Oct. 2020.

[42] Y. Ruan, X. Zhang, S.-C. Liang, and C. Joe-Wong, "Towards flexible device participation in federated learning," in *Proc. AISTATS*, 2021, pp. 3403–3411.

[43] C. Ying, H. Jin, X. Wang, and Y. Luo, "Double insurance: Incentivized federated learning with differential privacy in mobile crowdsensing," in *Proc. IEEE SRDS*, Sep. 2020, pp. 81–90.

[44] R. Zeng, S. Zhang, J. Wang, and X. Chu, "FMore: An incentive scheme of multi-dimensional auction for federated learning in MEC," in *Proc. IEEE ICDCS*, Nov. 2020, pp. 278–288.

[45] J. Kang, Z. Xiong, D. Niyato, H. Yu, Y.-C. Liang, and D. I. Kim, "Incentive design for efficient federated learning in mobile networks: A contract theory approach," in *Proc. IEEE VTS APWCS*, Aug. 2019, pp. 1–5.

[46] Y. Zhan, P. Li, Z. Qu, D. Zeng, and S. Guo, "A learning-based incentive mechanism for federated learning," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6360–6368, Jul. 2020.

[47] M. Shayan, C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Biscotti: A blockchain system for private and secure federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1513–1525, Jul./Aug. 2021.

[48] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Mitigating Sybils in federated learning poisoning," *CoRR*, vol. abs/1808.04866, pp. 1–16, Aug. 2018.

[49] K. Toyoda and A. N. Zhang, "Mechanism design for an incentive-aware blockchain-enabled federated learning platform," in *Proc. IEEE BigData*, Dec. 2019, pp. 395–403.

[50] M. H. U. Rehman, K. Salah, E. Damiani, and D. Svetinovic, "Towards blockchain-based reputation-aware federated learning," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Jul. 2020, pp. 183–188.

[51] N. B. Somy et al., "Ownership preserving AI market places using blockchain," in *Proc. IEEE Int. Conf. Blockchain*, Jul. 2019, pp. 156–165.

[52] Y. Zhang, L. Liu, Y. Gu, D. Niyato, M. Pan, and Z. Han, "Offloading in software defined network at edge with information asymmetry: A contract theoretical approach," *J. Signal Process. Syst.*, vol. 83, no. 2, pp. 241–253, May 2016.

[53] Z. Xiong, S. Feng, W. Wang, D. Niyato, P. Wang, and Z. Han, "Cloud/fog computing resource management and pricing for blockchain networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4585–4600, Jun. 2019.

[54] Z. Xiong, Y. Zhang, D. Niyato, P. Wang, and Z. Han, "When mobile blockchain meets edge computing," *IEEE Commun. Mag.*, vol. 56, no. 8, pp. 33–39, Aug. 2018.

[55] D. Zhang, F. R. Yu, and R. Yang, "Blockchain-based multi-access edge computing for future vehicular networks: A deep compressed neural network approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 12161–12175, Aug. 2022.

[56] N. H. Tran, W. Bao, A. Zomaya, M. N. H. Nguyen, and C. S. Hong, "Federated learning over wireless networks: Optimization model design and analysis," in *Proc. IEEE INFOCOM*, Apr. 2019, pp. 1387–1395.

[57] C. A. Hans, P. Sopasakis, J. Raisch, C. Reincke-Collon, and P. Patrinos, "Risk-averse model predictive operation control of islanded microgrids," *IEEE Trans. Control Syst. Technol.*, vol. 28, no. 6, pp. 2136–2151, Nov. 2020.

[58] D. Rosewater, R. Baldick, and S. Santoso, "Risk-averse model predictive control design for battery energy storage systems," *IEEE Trans. Smart Grid*, vol. 11, no. 3, pp. 2014–2022, May 2020.

[59] J. Kang, Z. Xiong, D. Niyato, S. Xie, and J. Zhang, "Incentive mechanism for reliable federated learning: A joint optimization approach to combining reputation and contract theory," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10700–10714, Dec. 2019.

[60] Z. Zhou, P. Liu, J. Feng, Y. Zhang, S. Mumtaz, and J. Rodriguez, "Computation resource allocation and task assignment optimization in vehicular fog computing: A contract-matching approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3113–3125, Apr. 2019.

[61] P. Dasgupta and E. Maskin, "The existence of equilibrium in discontinuous economic games. Part I: Theory," *Rev. Econ. Stud.*, vol. 53, pp. 1–26, Jan. 1986.

[62] J. Gao, L. Zhao, and X. Shen, "Network utility maximization based on an incentive mechanism for truthful reporting of local information," *IEEE Trans. Veh. Technol.*, vol. 67, no. 8, pp. 7523–7537, Aug. 2018.

[63] J. Kang, Z. Xiong, D. Ye, D. I. Kim, J. Zhao, and D. Niyato, "Toward secure blockchain-enabled internet of vehicles: Optimizing consensus management using reputation and contract theory," *IEEE Trans. Veh. Technol.*, vol. 68, no. 3, pp. 2906–2920, Mar. 2019.

[64] T. Liu, J. Li, F. Shu, M. Tao, W. Chen, and Z. Han, "Design of contract-based trading mechanism for a small-cell caching system," *IEEE Trans. Wireless Commun.*, vol. 16, no. 10, pp. 6602–6617, Oct. 2017.

[65] A. Ijaz et al., "Enabling massive IoT in 5G and beyond systems: PHY radio frame design considerations," *IEEE Access*, vol. 4, pp. 3322–3339, 2016.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.



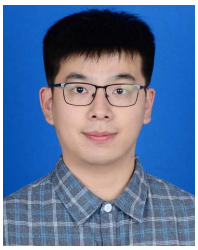
Xiaofei Wang (Senior Member, IEEE) received the B.S. degree from the Huazhong University of Science and Technology, China, and the M.S. and Ph.D. degrees from Seoul National University, Seoul, South Korea. He was a Post-Doctoral Fellow at The University of British Columbia, Vancouver, Canada, from 2014 to 2016. He is currently a Professor with the Tianjin Key Laboratory of Advanced Networking, College of Intelligence and Computing, Tianjin University, Tianjin, China. He has published more than 150 technical papers in IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS (JSAC), IEEE TRANSACTIONS ON CLOUD COMPUTING (TCC), IEEE/ACM TRANSACTIONS ON NETWORKING (ToN), IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS (TWC), IEEE INTERNET OF THINGS JOURNAL (IoTJ), IEEE COMMUNICATIONS SURVEYS AND TUTORIALS (COMST), IEEE TRANSACTIONS ON MULTIMEDIA (TMM), IEEE International Conference on Computer Communications (INFOCOM), and IEEE International Conference on Distributed Computing Systems (ICDCS). His research interests include edge computing, edge intelligence, and edge systems. In 2017, he was a recipient of the "Fred W. Ellersick Prize" from the IEEE Communication Society.



Yunfeng Zhao received the B.S. degree in information and computing science from Nanjing Agricultural University, China, in 2017, and the M.S. degree in probability and mathematical statistics from Tianjin University, China, in 2020, where she is currently pursuing the Ph.D. degree in computer science. Her current research interests include edge computing, distributed machine learning, and game theory. She received the B.S. National Scholarship of China in 2015, the Outstanding B.S. Graduates in 2017, and the M.S. National Scholarship of China in 2019.



Chao Qiu (Member, IEEE) received the B.S. degree in communication engineering from China Agricultural University in 2013 and the Ph.D. degree in information and communication engineering from the Beijing University of Posts and Telecommunications in 2019. From September 2017 to September 2018, she visited Carleton University, Ottawa, Ontario, Canada, as a Visiting Scholar. She is currently a Lecturer with the School of Computer Science and Technology, College of Intelligence and Computing, Tianjin University. Her current research interests include machine learning, software defined networking, and blockchain.



Zhicheng Liu received the B.S. degree in information security from Guizhou University, China, in 2015, and the M.S. degree in computer science and technology from Inner Mongolia University, China, in 2019. Currently, he is pursuing the Ph.D. degree in computer science with the College of Intelligence and Computing, Tianjin University, China. His research interests include edge computing, multi-agent learning, and game theory.



Jiangtian Nie (Member, IEEE) received the B.Eng. degree in electronics and information engineering from the Huazhong University of Science and Technology, Wuhan, China, and the Ph.D. degree from ERI@N, Interdisciplinary Graduate School, Nanyang Technological University (NTU), Singapore. She is currently a Research Fellow with NTU, Singapore. Her research interests include network economics, game theory, crowd sensing, and learning. She is the Editor of *Computer Networks*, *Computer Communications*, *Physical Communication*, and *EURASIP Journal on Wireless Communications and Networking*.



Victor C. M. Leung (Life Fellow, IEEE) is a Distinguished Professor of computer science and software engineering with Shenzhen University. He was a Professor of electrical and computer engineering and the TELUS Mobility Research Chair at the University of British Columbia (UBC) when he retired from UBC in 2018 and became a Professor Emeritus. He has coauthored more than 1300 journals/conference papers and book chapters. His research interests include wireless networks and mobile systems. He received the IEEE Vancouver Section Centennial Award, the 2011 UBC Killam Research Prize, the 2017 Canadian Award for Telecommunications Research, and the 2018 IEEE TCGCC Distinguished Technical Achievement Recognition Award. He coauthored papers that won the 2017 IEEE ComSoc Fred W. Ellersick Prize, the 2017 IEEE Systems Journal Best Paper Award, the 2018 IEEE CSIM Best Journal Paper Award, and the 2019 IEEE TCGCC Best Journal Paper Award. He is a fellow of the Royal Society of Canada, Canadian Academy of Engineering, and Engineering Institute of Canada. He is named in the current Clarivate Analytics list of "Highly Cited Researchers". He is on the Editorial Boards of the IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING, IEEE TRANSACTIONS ON CLOUD COMPUTING, IEEE ACCESS, *IEEE Network*, and several other journals.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:05:12 UTC from IEEE Xplore. Restrictions apply.